Министерство образования Республики Беларусь
Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет информационных технологий и управления
Кафедра информационных технологий автоматизированных систем

Дисциплина: Интернет-технологии и веб-программирование

ОТЧЁТ

по контрольной работе №1
«Серверная разработка»

Вариант № 37

Выполнил: ст. гр. 320602
М.В. Петюк

Проверил: Н. В. Хаджинова

Минск 2026

СОДЕРЖАНИЕ

1 Цель работы 3
2 Предметная область и сущность 4
3 Схема базы данных 6
4 Реализация 8
4.1 Структура проекта и окружение 8
4.2 Эндпоинты 8
4.3 Аутентификация и права доступа 9
4.4 Валидация и обработка ошибок 10
4.5 Дополнительные эндпоинты 10
4.6 Документация *API* 11
5 Тестирование *API* 12
6 Репозиторий и коллекция *Postman* 15
7 Ответы на контрольные вопросы 16
Выводы 19
Список использованных источников 20
Приложение А (обязательное) Листинги моделей, маршрутов, *middleware* и валидации 21

1 ЦЕЛЬ РАБОТЫ

Разработать *REST API* на *Node.js* с аутентификацией *JWT*, работой с базой данных *PostgreSQL* и *CRUD*-операциями для основной сущности курсового проекта «Система управления складом и отгрузкой товаров (*WMS*) с *QR*-кодами».

2 ПРЕДМЕТНАЯ ОБЛАСТЬ И СУЩНОСТЬ

Курсовой проект – система управления складом и отгрузкой товаров. Склад принимает товар, размещает его по ячейкам хранения и отгружает покупателям, а каждая операция подтверждается сканированием *QR*-кода. Основной сущностью контрольной работы выбрана отгрузка (модель *Shipment*, таблица *shipments*) – заказ покупателя, который собирают, упаковывают и отправляют со склада. Это центральный процесс темы: в названии проекта и отгрузка товаров, и *QR*-коды.

Каждой отгрузке сервер выдаёт уникальный *QR*-код. Он печатается на этикетке грузового места: кладовщик сканирует этикетку терминалом, получает карточку отгрузки и переводит её в следующий статус – сборка, упаковка, отправка. Пример *QR*-кода, сформированного *API*, приведён на рисунке 1.



Рисунок 1 – Пример *QR*-кода этикетки отгрузки *SHP-5970B8CD152DA8F6*

Работа продолжает серверную часть из индивидуальной практической работы №1, где уже были пользователи, роли, справочники товаров и ячеек и журнал аудита. В контрольной работе добавлены таблица *shipments* и маршруты */shipments*.

В задании маршруты названы */items*. В проекте используется */shipments* – название сущности предметной области; состав маршрутов и правила доступа соответствуют заданию (таблица 1).

Таблица 1 – Соответствие маршрутов задания и *API*

Маршрут задания	Маршрут *API*	Доступ
GET /items	*GET /shipments*	после входа
GET /items/:id	*GET /shipments/:id*	после входа
POST /items	*POST /shipments*	роль *user* и выше
PUT /items/:id	*PUT /shipments/:id*	автор или администратор
DELETE /items/:id	*DELETE /shipments/:id*	только *admin*
дополнительные эндпоинты	*GET /shipments*?… *GET /shipments/stats* *GET /shipments/scan/:code* *GET /shipments/:id/qr* *PATCH /shipments/:id/status*	после входа

Роли: *user* – кладовщик, *admin* – администратор. Между ними есть роль *moderator* – старший смены, который может изменять любую отгрузку, но не удалять её. Требование задания «автор или админ» выполняется: кладовщик меняет только свои отгрузки. Список отгрузок, в отличие от каталога товаров, без входа не отдаётся: в нём имена и адреса получателей.

Поля отгрузки приведены в таблице 2. Уникальных полей два: номер заказа *orderNumber*, который вводит кладовщик, и *QR*-код *qrCode*, который выдаёт сервер. Плановая дата не может быть в прошлом, количество мест – от 1 до 1000, вес не может быть отрицательным.

Таблица 2 – Поля сущности «Отгрузка»

Поле	Тип	Правила
orderNumber	строка до 32	обязательное, уникальное, латиница, цифры и дефис, хранится в верхнем регистре
qrCode	строка	выдаёт сервер: *SHP*- и 16 шестнадцатеричных символов, уникальное
customerName	строка 2–200	обязательное, получатель
deliveryAddress	строка 5–500	обязательное
plannedDate	дата	обязательное, ГГГГ-ММ-ДД, не раньше сегодняшнего дня
status	строка	*new* при создании, меняется отдельным маршрутом
packagesCount	целое	от 1 до 1000, по умолчанию 1
totalWeightKg	число, 3 знака	необязательное, от 0 до 50 000
carrier	строка до 100	необязательное, перевозчик
comment	текст до 2000	необязательное
shippedAt	дата и время	ставит сервер при отправке
createdBy	*UUID*	автор, сервер берёт из токена

Статус отгрузки меняется только вперёд по цепочке складских операций (таблица 3). Отменить отгрузку можно, пока груз не покинул склад. После отправки карточка становится документом и не редактируется.

Таблица 3 – Допустимые переходы статуса

Статус	Значение	Можно перевести в
new	создана	*picking*, *cancelled*
picking	идёт сборка	*packed*, *cancelled*
packed	упакована	*shipped*, *cancelled*
shipped	отправлена	*delivered*
delivered	доставлена	–
cancelled	отменена	–

3 СХЕМА БАЗЫ ДАННЫХ

Используется *PostgreSQL* 16 и *ORM Sequelize* 6. Таблицы создаются из моделей при запуске сервера и командой *npm run seed*: функция *syncSchema* создаёт недостающие таблицы и добавляет новые столбцы, не удаляя данные. Поэтому база, созданная для первой работы, дополнилась таблицей *shipments* без потери записей. Для контрольной работы основные таблицы – *users* и *shipments* (таблицы 4 и 5).

Таблица 4 – Таблица *users*

Столбец	Тип	Ограничения
id	*UUID*	первичный ключ
email	*VARCHAR(255)*	*NOT NULL*, *UNIQUE*
passwordHash	*VARCHAR(255)*	*NOT NULL*, *bcrypt*-хеш
role	*VARCHAR(20)*	*NOT NULL*, *user*, *moderator* или *admin*, по умолчанию '*user*'
name	*VARCHAR(100)*	*NULL*
failedAttempts	*INTEGER*	*NOT NULL*, по умолчанию 0
lockUntil	*TIMESTAMPTZ*	*NULL*
createdAt, *updatedAt*	*TIMESTAMPTZ*	*NOT NULL*

Таблица 5 – Таблица *shipments*

Столбец	Тип	Ограничения
id	*UUID*	первичный ключ
orderNumber	*VARCHAR(32)*	*NOT NULL*, *UNIQUE*
qrCode	*VARCHAR(32)*	*NOT NULL*, *UNIQUE*
customerName	*VARCHAR(200)*	*NOT NULL*
deliveryAddress	*VARCHAR(500)*	*NOT NULL*
plannedDate	*DATE*	*NOT NULL*, индекс
status	*VARCHAR(16)*	*NOT NULL*, по умолчанию '*new*', индекс
packagesCount	*INTEGER*	*NOT NULL*, по умолчанию 1
totalWeightKg	*NUMERIC*(10,3)	*NULL*
carrier	*VARCHAR(100)*	*NULL*
comment	*TEXT*	*NULL*
shippedAt	*TIMESTAMPTZ*	*NULL*
createdBy	*UUID*	*NULL*, внешний ключ на *users.id*
createdAt, *updatedAt*	*TIMESTAMPTZ*	*NOT NULL*

Связи между таблицами:

– *users – shipments*: один ко многим по *shipments.createdBy*, при удалении пользователя автор обнуляется (*ON DELETE SET NULL*), а отгрузка остаётся в истории склада;

– *users – sessions*: один ко многим по *sessions.userId*, сессии удаляются вместе с пользователем (*ON DELETE CASCADE*);

– *users – products*, *users – storage_locations*, *users – audit_logs*: один ко многим, *ON DELETE SET NULL*.

Текстовая схема:

```
users (id PK, email UQ, passwordHash, role, name, ...)
  1 ──< shipments (id PK, orderNumber UQ, qrCode UQ,
                   customerName, deliveryAddress,
                   plannedDate IDX, status IDX, packagesCount,
                   totalWeightKg, carrier, comment, shippedAt,
                   createdBy FK → users.id)
  1 ──< sessions (id PK, userId FK → users.id, tokenHash UQ, ...)
  1 ──< products (id PK, sku UQ, createdBy FK → users.id, ...)
  1 ──< storage_locations (createdBy FK → users.id)
  1 ──< audit_logs (userId FK → users.id)
```

Индексы по *status* и *plannedDate* ускоряют самые частые запросы склада: отгрузки на сегодня и отгрузки в нужном статусе. Столбец *NUMERIC* драйвер *pg* возвращает строкой, поэтому в модели для *totalWeightKg* задан геттер, и в *JSON*-ответе вес приходит числом. Листинги моделей приведены в приложении А.

4 РЕАЛИЗАЦИЯ

4.1 Структура проекта и окружение

Сервер написан на *Node.js* и *Express* 5. Код разделён на слои: *src/models* – модели *Sequelize*, *src/routes* – маршруты, *src/middleware* – аутентификация, проверка роли, валидация и обработка ошибок, *src/validators* – схемы *Joi*, *src/controllers* и *src/services* – логика запросов, *src/constants* – статусы отгрузок и допустимые переходы, *src/docs* – описание *OpenAPI*.

Основные зависимости из задания: *express*, *jsonwebtoken*, *bcrypt*, *helmet*, *express-rate-limit*, *sequelize*, *pg*, *pg-hstore*, *nodemon*. Дополнительно подключены *joi* (валидация), *qrcode* (изображения *QR*-кодов), *cors*, *cookie-parser*, *winston* (журналы), *swagger-ui-express* (документация), *jest* и *supertest* (тесты).

Настройки читаются из файла *.env*: *PORT* – порт сервера, *DATABASE_URL* – строка подключения к *PostgreSQL*, *JWT_SECRET* – секрет подписи *access*-токена, а также *JWT_REFRESH_SECRET*, *CORS_ORIGINS* и учётная запись администратора для *npm run seed*. Файл *.env* добавлен в *.gitignore*, в репозитории лежит только *.env.example* с примерами значений. Если *JWT_SECRET* короче 32 символов или *DATABASE_URL* не задан, сервер не запускается.

4.2 Эндпоинты

Эндпоинты аутентификации и отгрузок приведены в таблице 6. Полный список, включая пользователей, товары, ячейки и журнал аудита, описан в *Swagger* и в файле *API.md*.

Таблица 6 – Эндпоинты аутентификации и отгрузок

Метод и путь	Назначение	Доступ	Коды
POST /auth/register	регистрация	все	201, 400, 409
POST /auth/login	вход, выдача *JWT*	все	200, 400, 401, 423
GET /auth/me	профиль по токену	после входа	200, 401
GET /shipments	список с фильтрами, поиском, сортировкой	после входа	200, 400, 401
GET /shipments/stats	статистика	после входа	200, 401
GET /shipments/scan/:code	поиск по *QR*-коду	после входа	200, 400, 401, 404
GET /shipments/:id	карточка	после входа	200, 400, 401, 404
GET /shipments/:id/qr	изображение *QR*-кода	после входа	200, 401, 404
POST /shipments	создание	*user* и выше	201, 400, 401, 409
PUT /shipments/:id	полная замена	автор, *moderator*, *admin*	200, 400, 401, 403, 404, 409
PATCH /shipments/:id/status	смена статуса	после входа	200, 400, 401, 404, 409
DELETE /shipments/:id	удаление	*admin*	200, 401, 403, 404

4.3 Аутентификация и права доступа

POST /auth/register проверяет тело схемой *Joi*, приводит *email* к нижнему регистру, проверяет, что такой *email* ещё не зарегистрирован, хеширует пароль *bcrypt* с 12 раундами и создаёт пользователя с ролью *user*. Поле *role* из тела отбрасывается, поэтому назначить себе роль администратора при регистрации нельзя.

POST /auth/login находит пользователя по *email* и сравнивает пароль функцией *bcrypt.compare*. При успехе выдаётся *access*-токен *JWT* (алгоритм *HS256*, срок 15 минут) с полями *sub* – идентификатор пользователя, *role* и *sid* – идентификатор сессии. Токен передаётся в заголовке *Authorization*: *Bearer <token>*. Дополнительно сервер ставит *refresh*-токен в *cookie HttpOnly* для продления сессии без повторного ввода пароля.

Middleware authenticate проверяет подпись и срок токена, загружает пользователя из базы и проверяет, что сессия не отозвана. Роль берётся из базы, поэтому смена роли действует сразу. Для маршрутов */shipments* он подключён на весь роутер. *Middleware roleGuard(role)* сравнивает уровень

роли пользователя с требуемым: *user* < *moderator* < *admin*, им защищены создание и удаление.

Право автора проверяет контроллер, так как для этого нужна запись из базы: отгрузку может менять тот, чей *id* записан в *createdBy*, а также *moderator* и *admin*. Иначе возвращается 403 «Изменять отгрузку может только её автор или администратор», а попытка записывается в журнал безопасности. Статус может менять любой вошедший сотрудник: собирает и упаковывает заказ не обязательно тот, кто его создал. Удаление выполняет только администратор, оно записывается в журнал аудита.

4.4 Валидация и обработка ошибок

Middleware validate(*schema*, *source*) проверяет тело, параметры пути или строку запроса схемой *Joi*. Все ошибки собираются сразу (*abortEarly*: *false*), лишние поля отбрасываются (*stripUnknown*), строки приводятся к нужным типам. Поэтому *status* и *qrCode* из тела *POST* игнорируются: статус всегда *new*, а код выдаёт сервер. Дата проверяется дважды: что она существует (2026-02-30 отклоняется) и что она не раньше сегодняшнего дня. Ошибка возвращается кодом 400 со списком полей. Пример ответа на *POST /shipments* с неверными данными:

```
{"error": {
  "message": "Ошибка валидации",
  "details": [
    {"field": "orderNumber",
     "message": "Номер заказа: 3–32 символа, латиница, цифры и дефис"},
    {"field": "plannedDate",
     "message": "Плановая дата отгрузки не может быть в прошлом"},
    {"field": "packagesCount", "message": "Количество мест не меньше 1"},
    {"field": "totalWeightKg", "message": "Вес не может быть
отрицательным"}
  ]
}}
```

Уникальность проверяется до записи: занятый номер заказа или *email* возвращают код 409 с указанием поля, уникальные индексы в базе страхуют от одновременных запросов. Код 409 возвращается и при недопустимом переходе статуса – в *details* указаны текущий статус, запрошенный и разрешённые. Остальные ошибки обрабатывает общий обработчик: 401 – нет или неверный токен, 403 – недостаточно прав, 404 – запись не найдена. Стек вызовов и текст *SQL* клиенту не отдаются.

4.5 Дополнительные эндпоинты

GET /shipments принимает параметры строки запроса: *search* – подстрока в номере заказа, получателе или адресе без учёта регистра, *status*, диапазон плановой даты *plannedFrom* и *plannedTo*, *sort* – поле сортировки, *order* –

направление, а также *page* и *limit*. Поле сортировки берётся только из белого списка, *plannedTo* не может быть раньше *plannedFrom*. По умолчанию отгрузки отсортированы по плановой дате – в порядке, в котором их нужно собирать. Пример:

```
GET /shipments?status=new&plannedFrom=2026-11-01&plannedTo=2026-11-07
```

GET /shipments/stats возвращает сводку для начальника смены: количество отгрузок по каждому статусу, незакрытые отгрузки на сегодня и просроченные, сумму грузовых мест и веса без отменённых. Подсчёт по статусам выполняется одним *SQL*-запросом с *GROUP BY*.

GET /shipments/:id/qr формирует изображение *QR*-кода библиотекой *qrcode*: *PNG* по умолчанию или *SVG* при *format=svg*. В *QR*-коде закодировано значение *qrCode*, а не *id* записи, поэтому по этикетке нельзя перебирать идентификаторы базы. *GET /shipments/scan/:code* ищет отгрузку по отсканированному коду; формат кода проверяется до запроса к базе.

4.6 Документация *API*

API описан в формате *OpenAPI* 3.0: интерактивная документация *Swagger UI* доступна по адресу *http://localhost:3000/api-docs*, *JSON*-описание – по адресу */api-docs.json*. В описании указаны схемы тела запросов, параметры фильтрации, переходы статусов и коды ответов. Дополнительно в репозитории есть файл *API.md* с примерами запросов и ответов и *README.md* с инструкцией по установке и запуску.

5 ТЕСТИРОВАНИЕ *API*

Перед проверкой база очищалась командой *npm run db:reset*, которая создаёт таблицы и администратора из *.env*, затем сервер запускался командой *npm run dev*. Коллекция *Postman WMS-KR1* состоит из шести папок, которые выполняются по порядку: аутентификация, *CRUD* отгрузок, статусы и *QR*-код, фильтрация и статистика, ошибки и права доступа, действия администратора. Токены, *id* и *QR*-код созданной отгрузки сохраняются в переменных коллекции скриптами на вкладке *Tests*, а плановые даты считаются от текущего дня.

На рисунке 2 показана регистрация кладовщика. В теле передано поле "*role*": "*admin*", но пользователь создан с ролью *user*, а пароль в ответ не попадает.

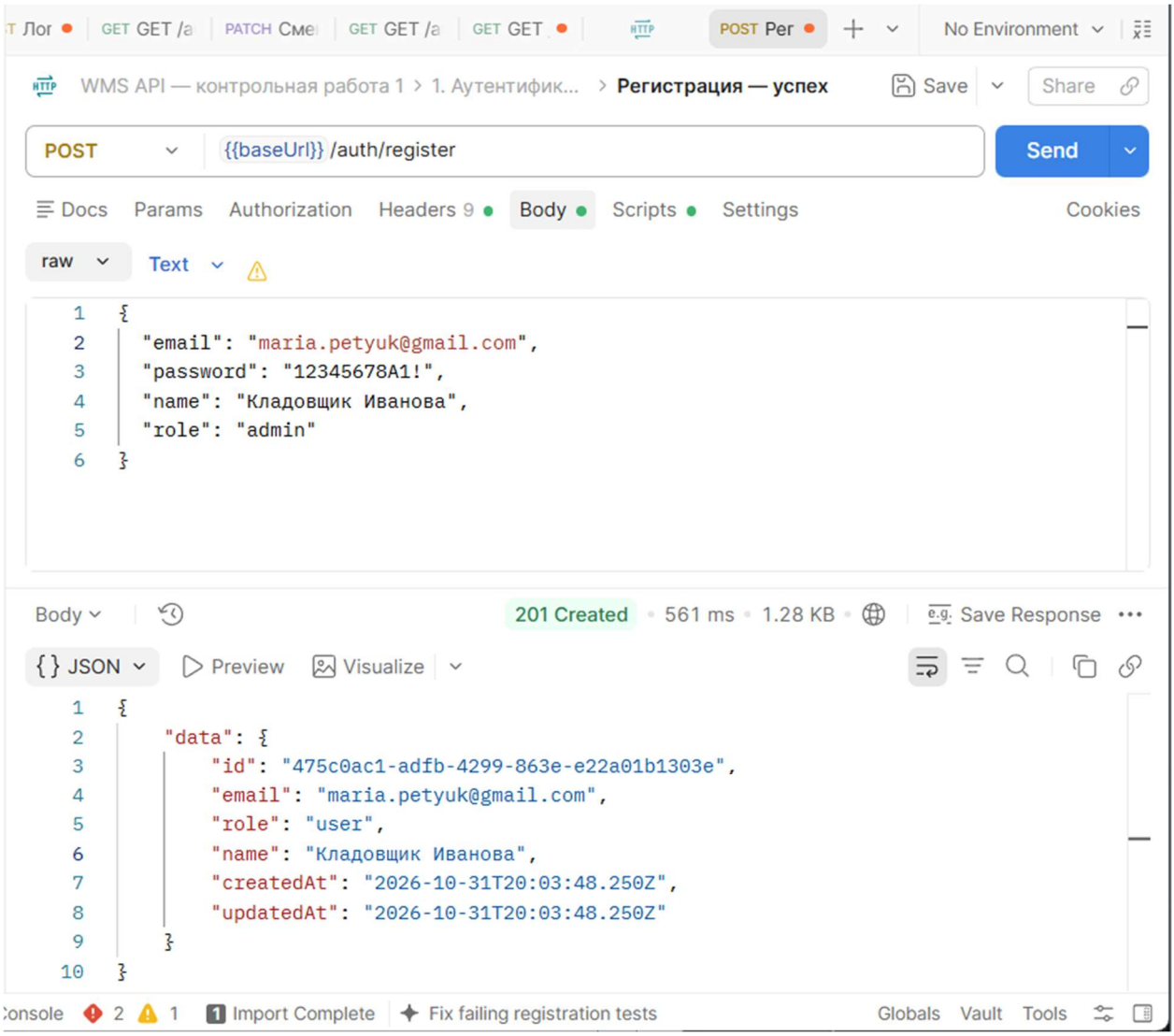


Рисунок 2 – Регистрация *POST /auth/register*

Повторная регистрация с тем же *email* отклоняется кодом 409 (рисунок 3).

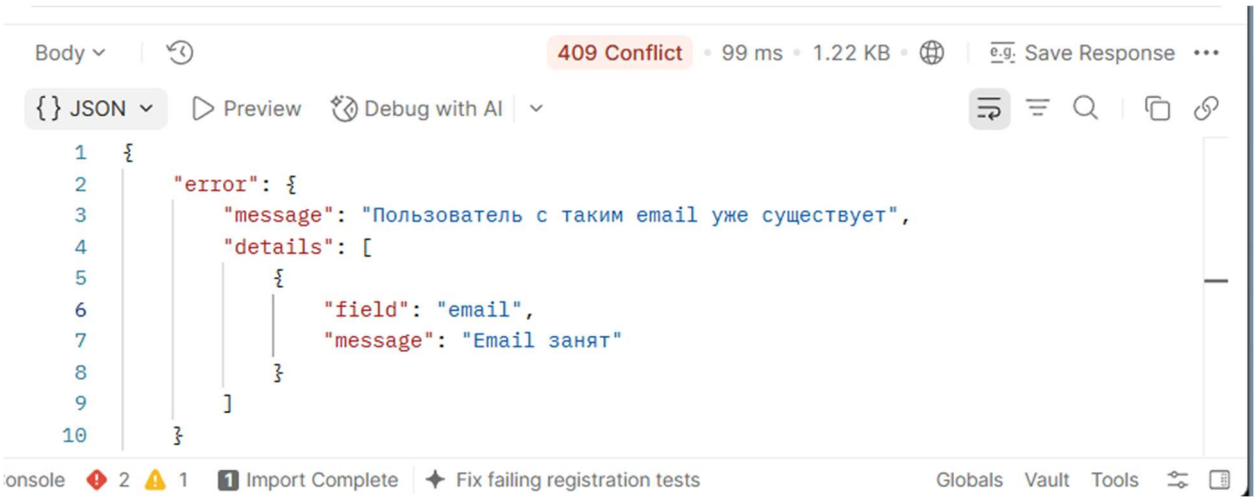


Рисунок 3 – Повторная регистрация с тем же *email*

На рисунке 4 показан вход: в ответе выдан *accessToken*, тип *Bearer* и срок действия 900 секунд. Скрипт *Tests* сохраняет токен в переменную *authorToken*.

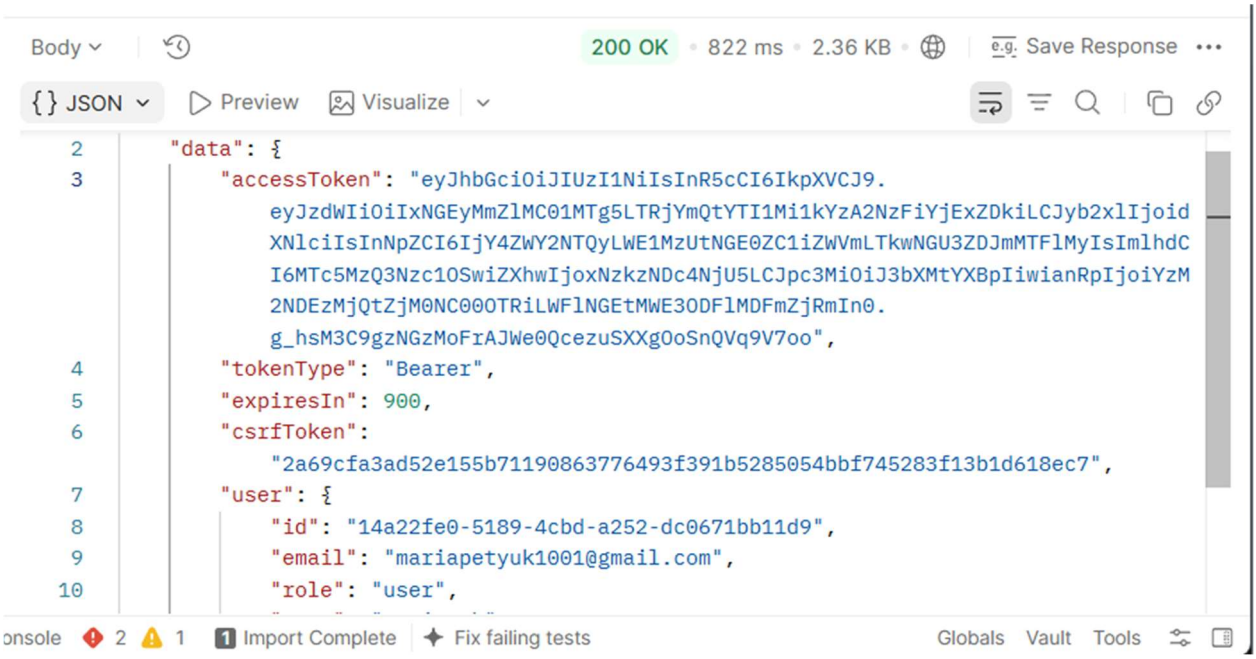


Рисунок 4 – Вход *POST /auth/login* и получение токена

Запрос с токеном в заголовке *Authorization* возвращает профиль текущего пользователя (рисунок 5).

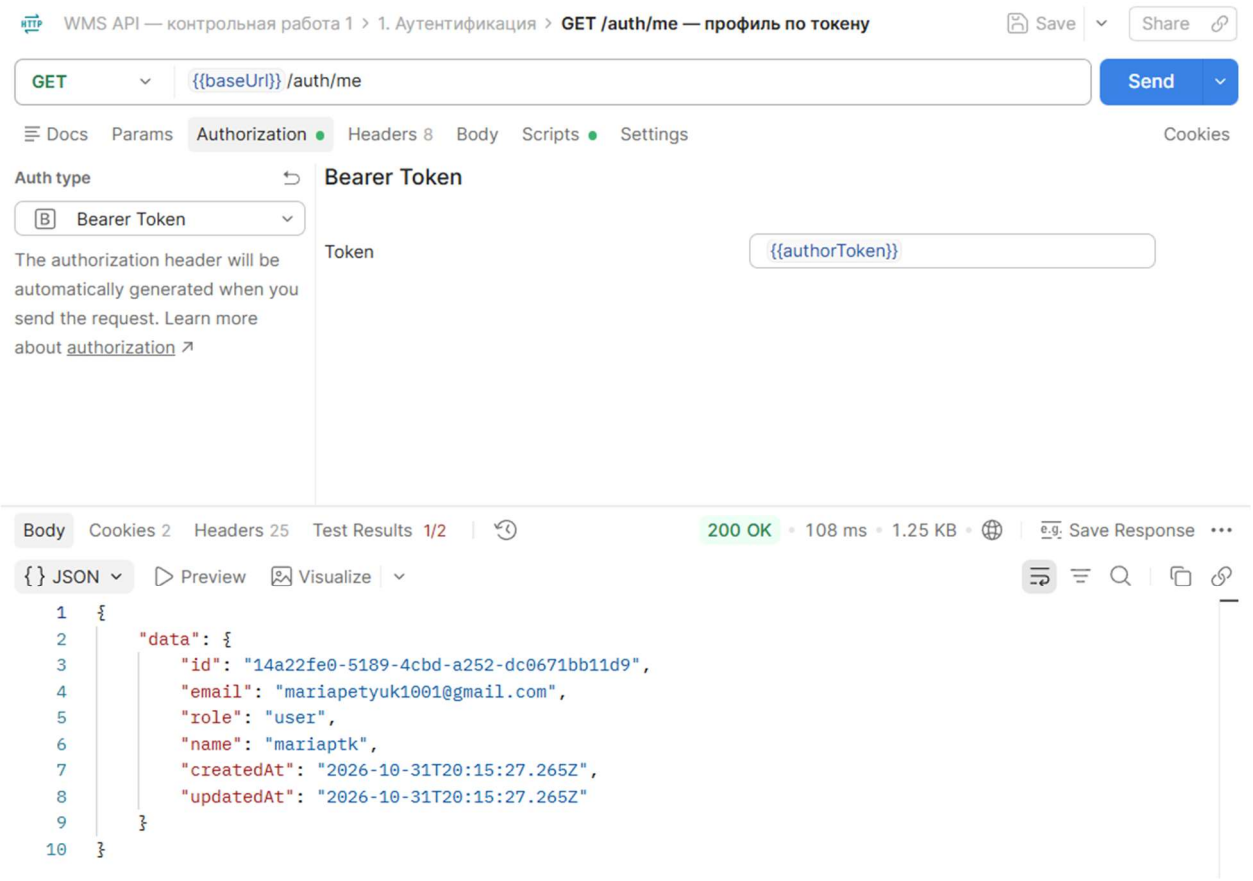


Рисунок 5 – Запрос с токеном *GET /auth/me*

Создание отгрузки показано на рисунке 6. В теле переданы "*status*": "*shipped*" и свой *qrCode*, но сервер их отбросил: отгрузка создана в статусе *new* с выданным сервером *QR*-кодом.

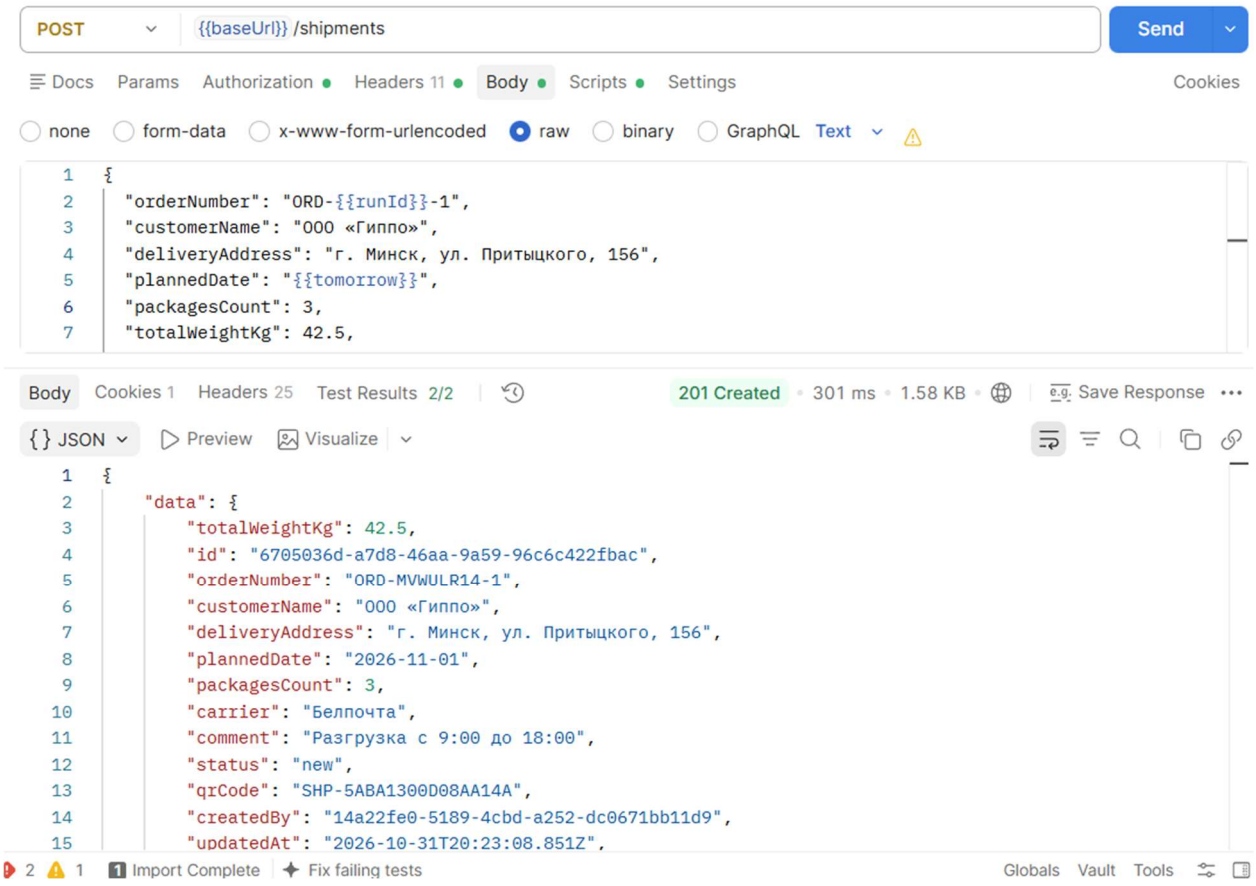


Рисунок 6 – Создание отгрузки *POST /shipments*

Список отгрузок с блоком *meta* показан на рисунке 7, карточка по *id* – на рисунке 8.

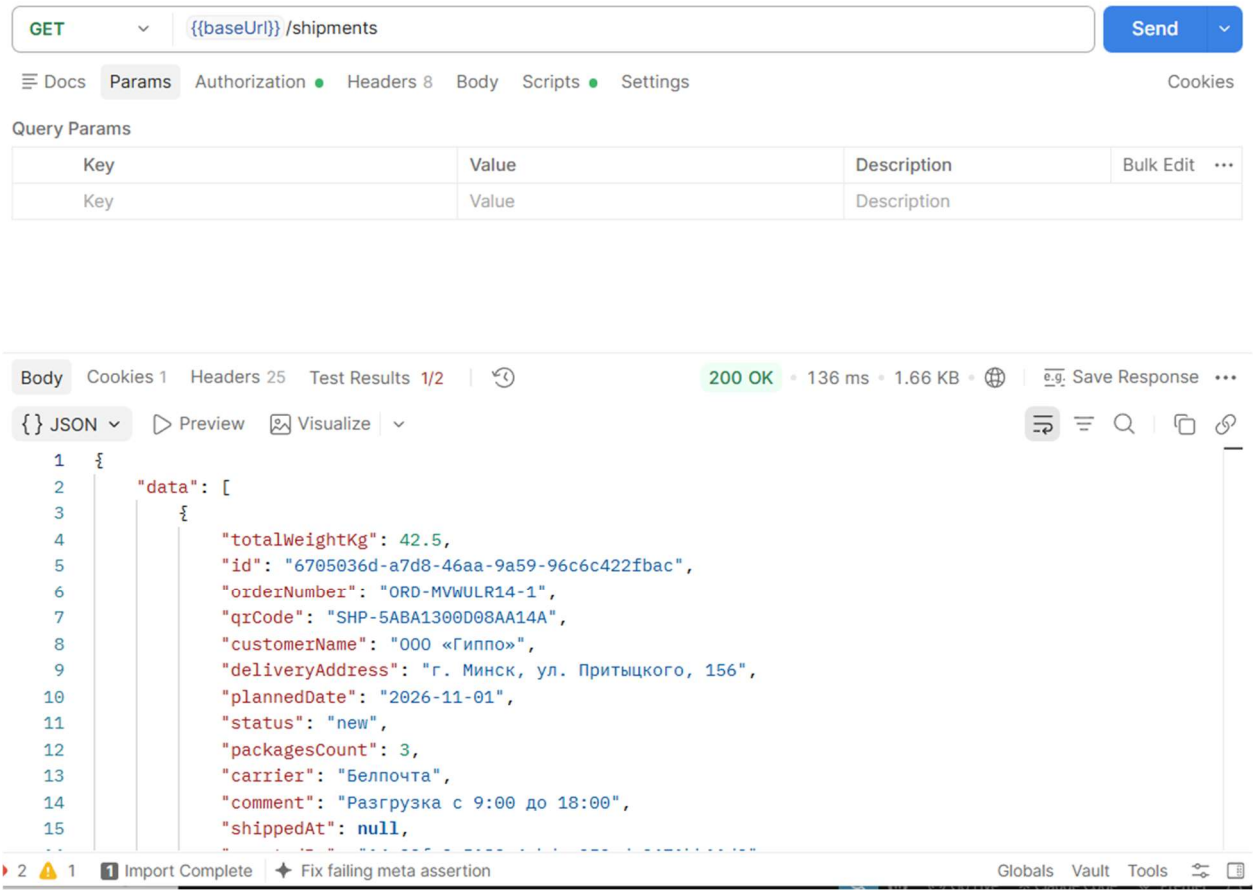


Рисунок 7 – Список отгрузок *GET /shipments*

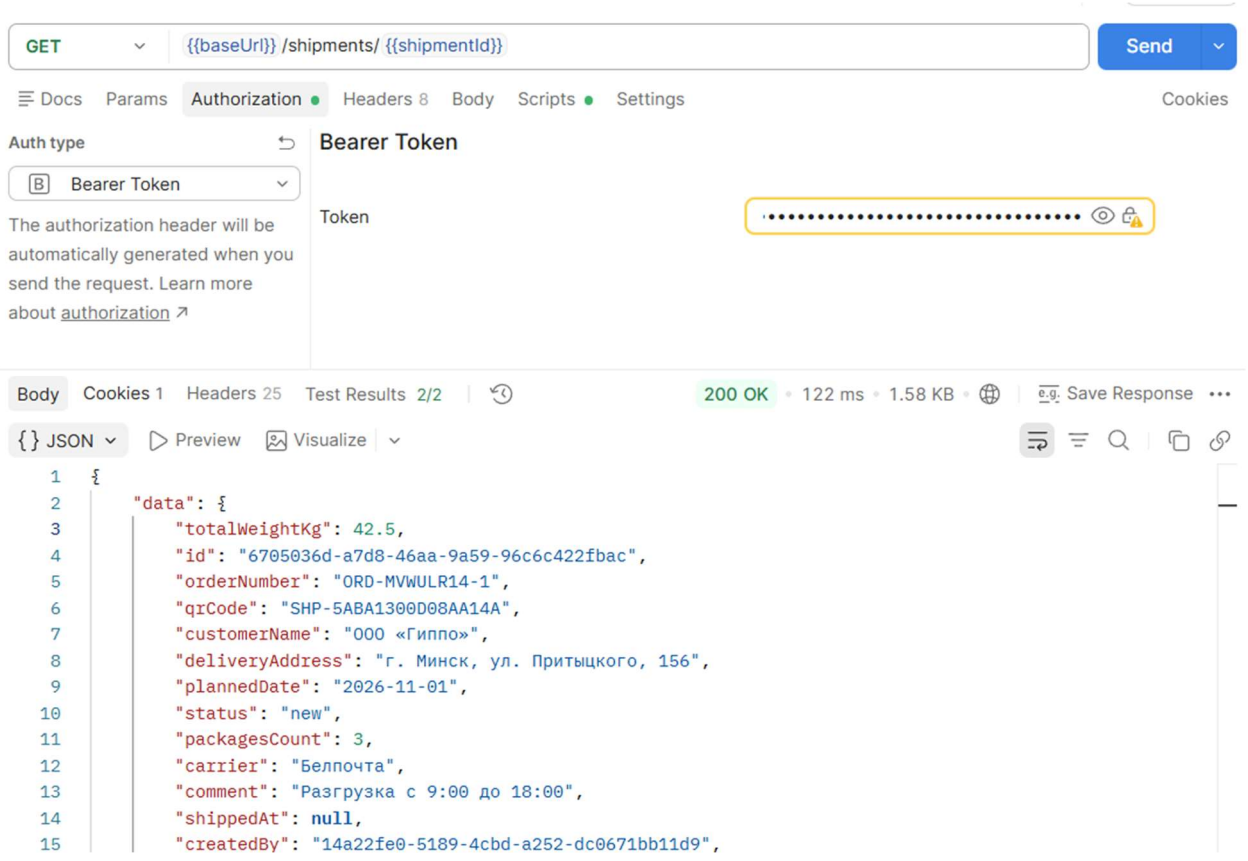


Рисунок 8 – Карточка отгрузки *GET /shipments/:id*

Автор заменяет карточку целиком запросом *PUT*, *QR*-код при этом не меняется (рисунок 9).

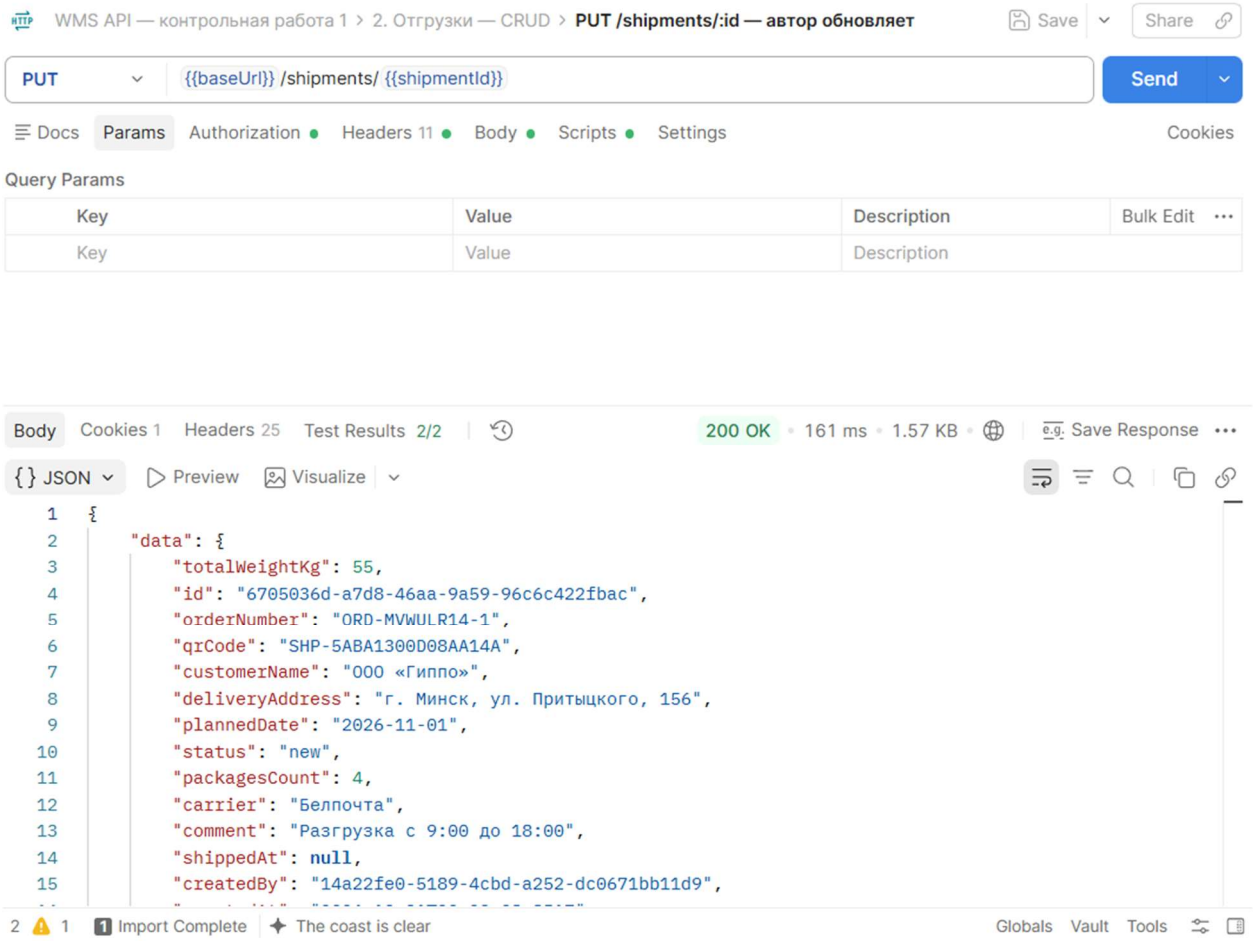


Рисунок 9 – Обновление отгрузки автором *PUT /shipments/:id*

Смена статуса показана на рисунке 10. Попытка перескочить из *packed* сразу в *delivered* отклоняется кодом 409 со списком разрешённых переходов (рисунок 11).

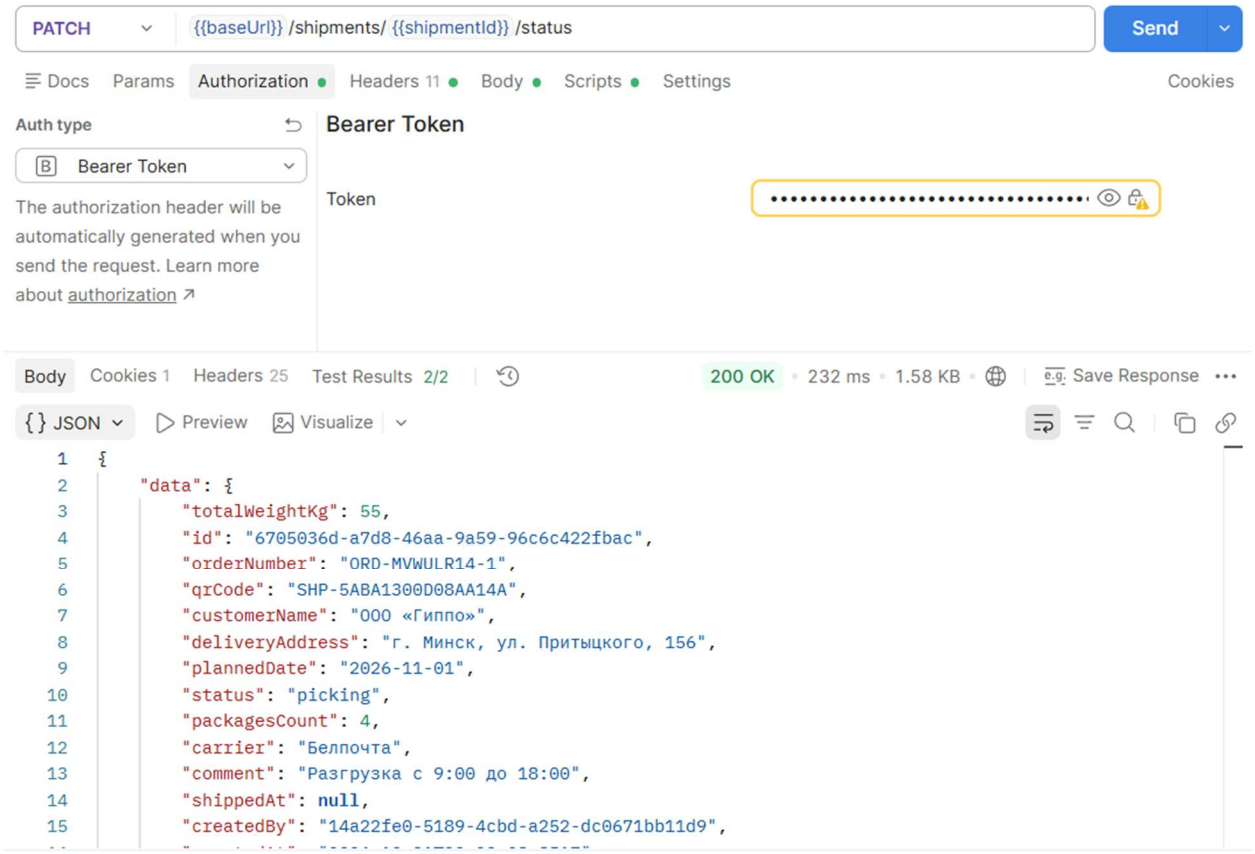


Рисунок 10 – Смена статуса *PATCH /shipments/:id/status*

На рисунке 12 показано изображение *QR*-кода для этикетки – *Postman* отображает *PNG* на вкладке ответа. Поиск отгрузки по этому коду, как при сканировании терминалом, показан на рисунке 13.

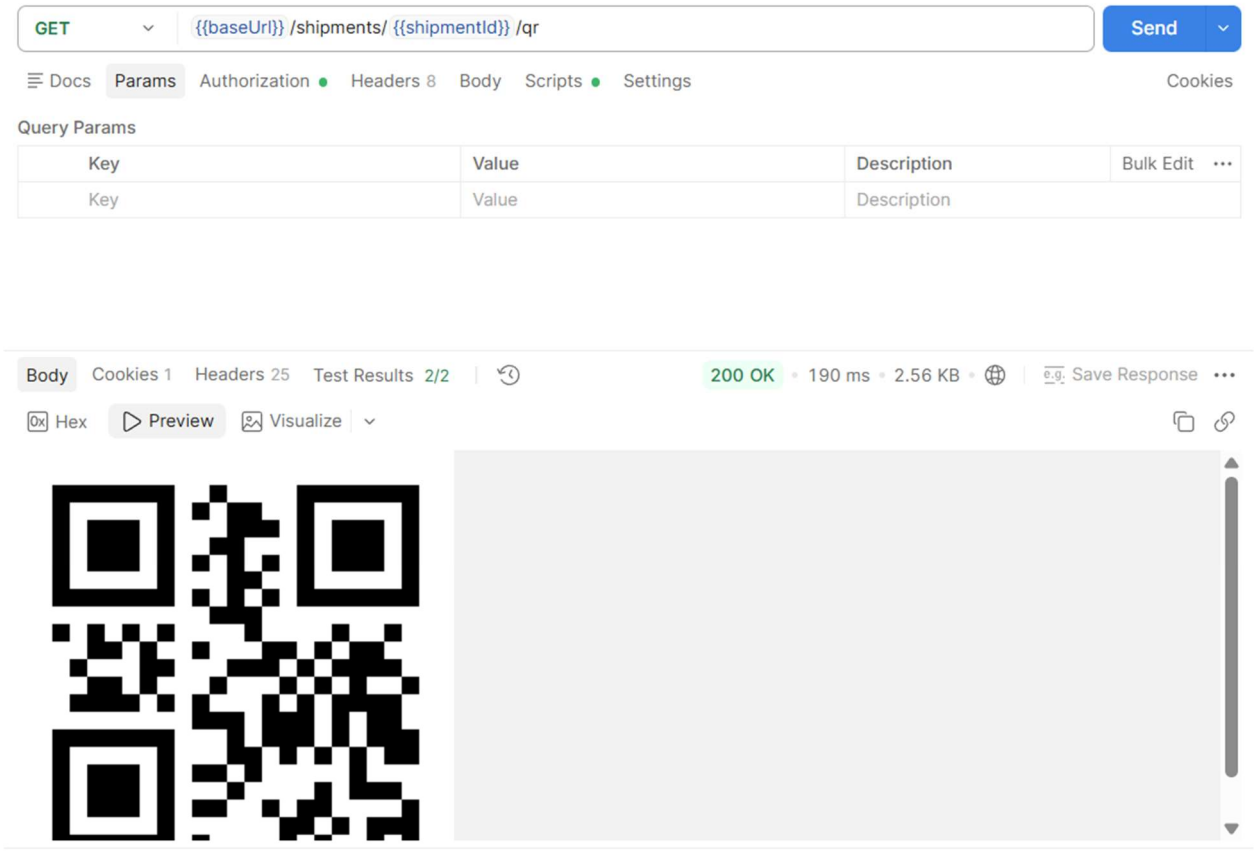


Рисунок 12 – *QR*-код отгрузки *GET /shipments/:id/qr*

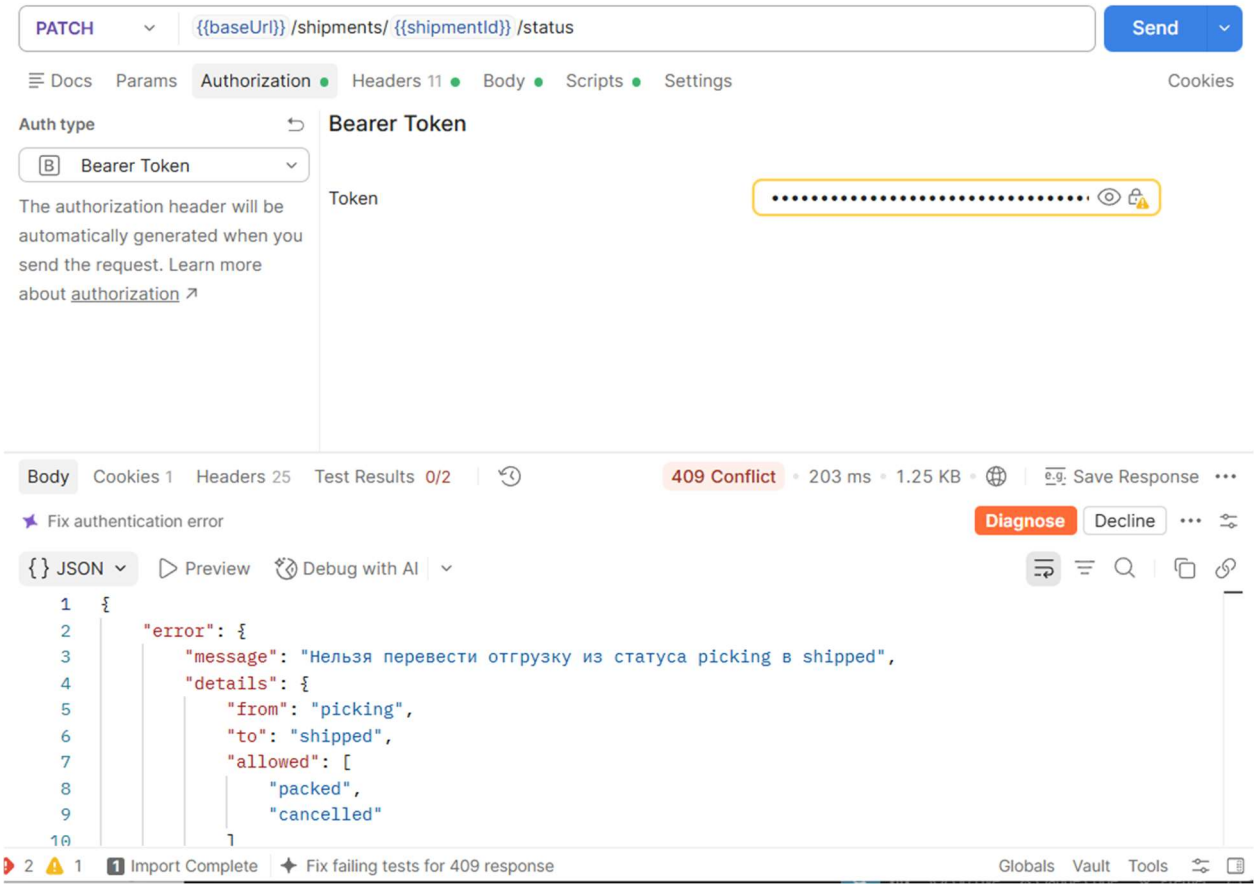


Рисунок 13 – Поиск по *QR*-коду *GET /shipments/scan/:code*

Фильтрация по диапазону плановой даты с сортировкой по номеру заказа показана на рисунке 14, статистика – на рисунке 15.

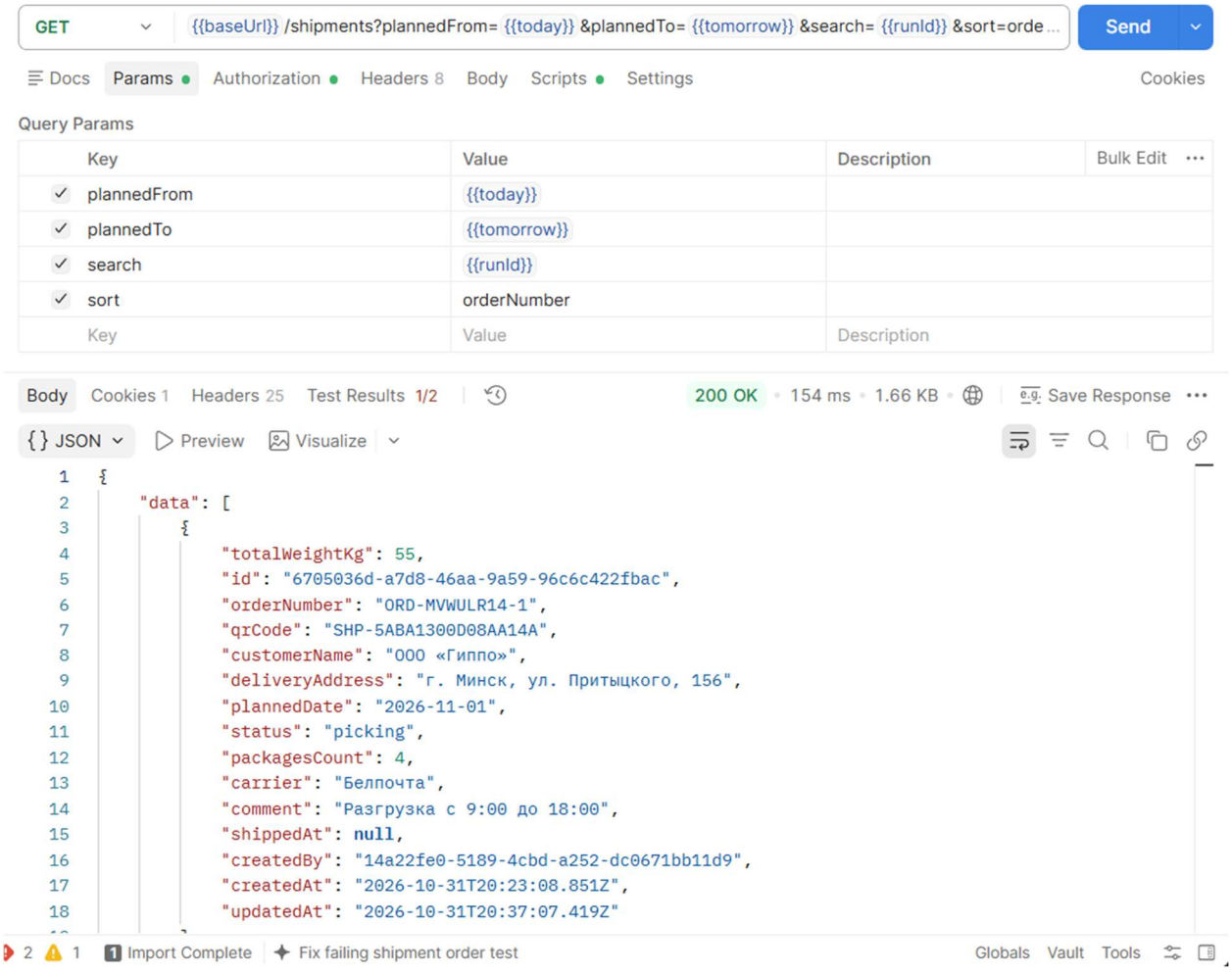


Рисунок 14 – Фильтр по дате и сортировка

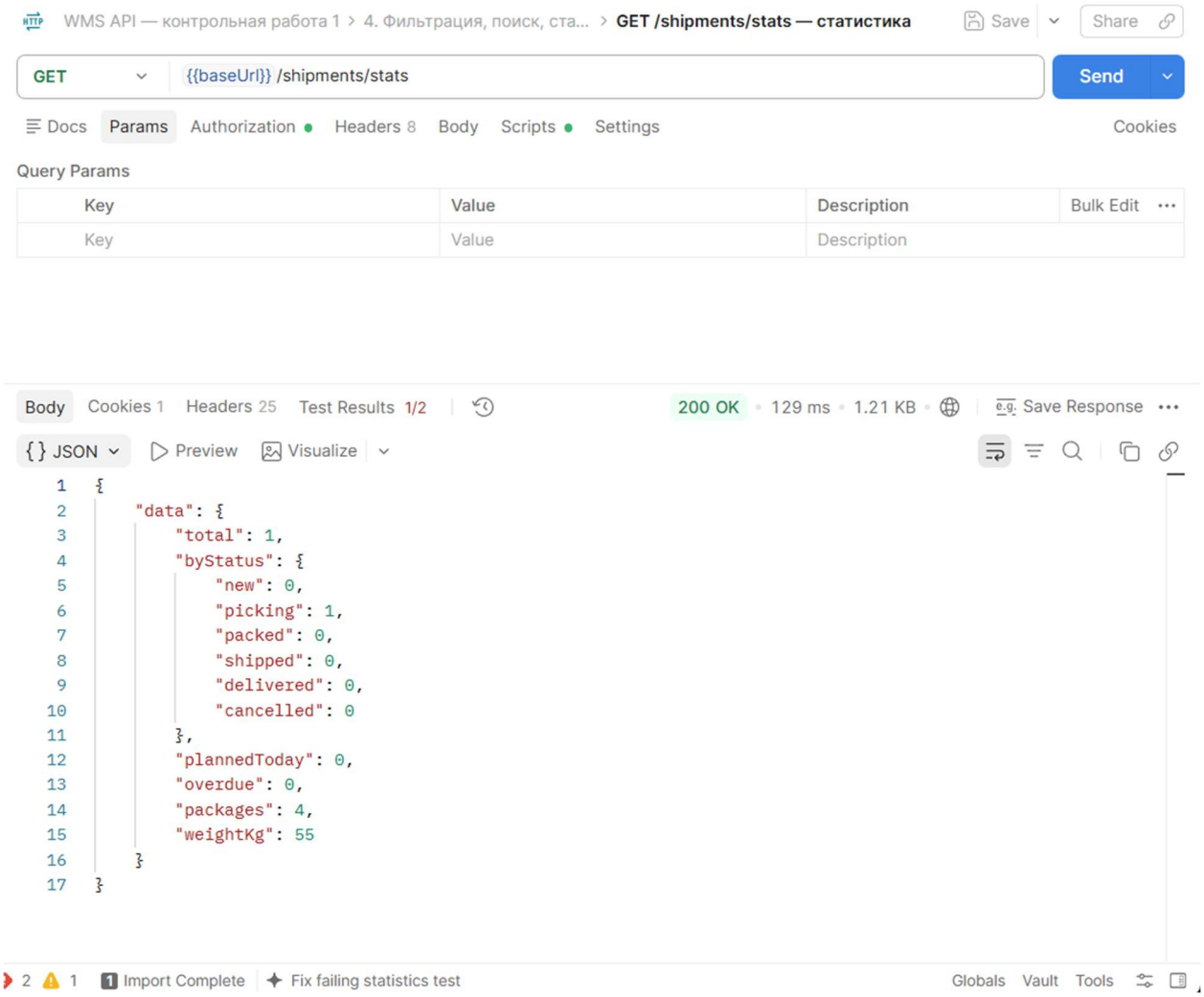


Рисунок 15 – Статистика *GET /shipments/stats*

Удалять отгрузки может только администратор: кладовщик получает код 403 (рисунок 16), администратор удаляет отгрузку (рисунок 17), после чего *GET* по этому *id* возвращает 404.

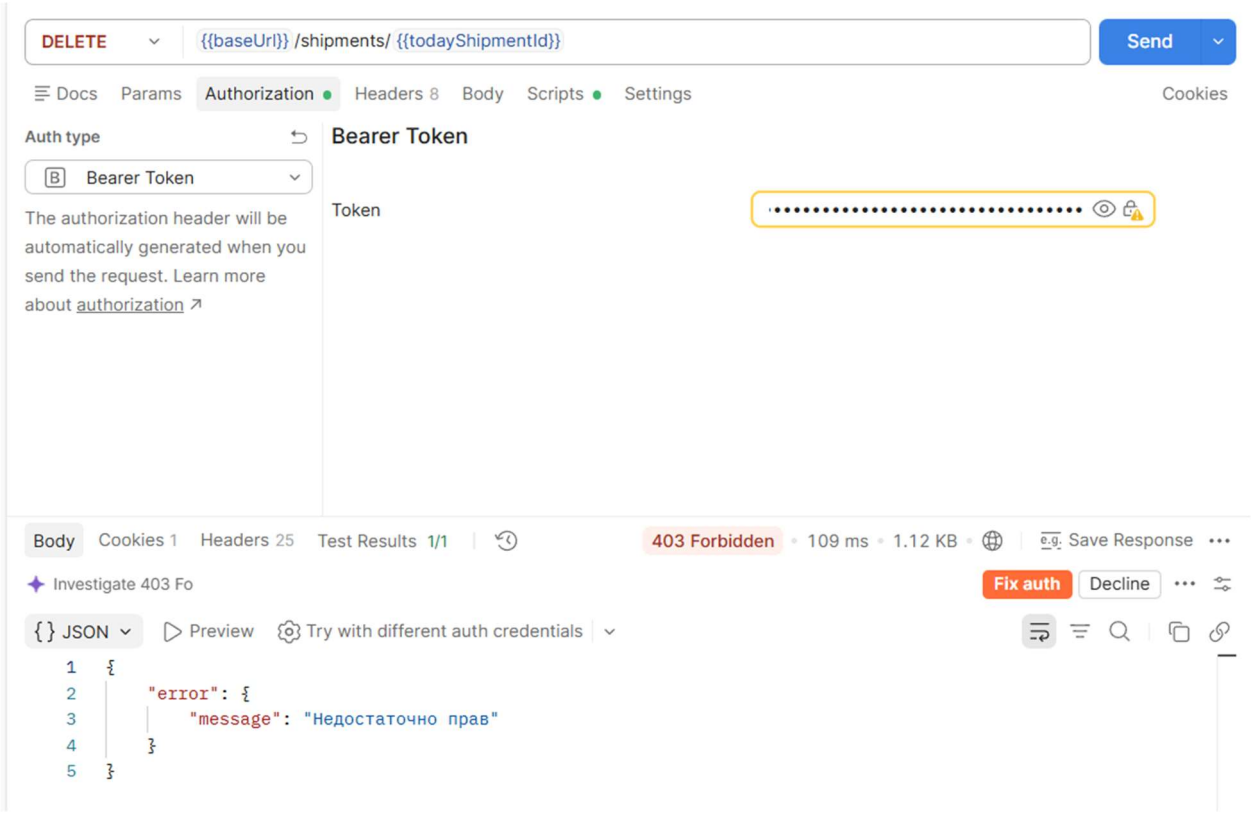


Рисунок 16 – Удаление отгрузки кладовщиком

Рисунок 18 – Ошибки валидации при создании отгрузки

Отгрузка с уже существующим номером заказа не создаётся, сервер отвечает 409 (рисунок 17).

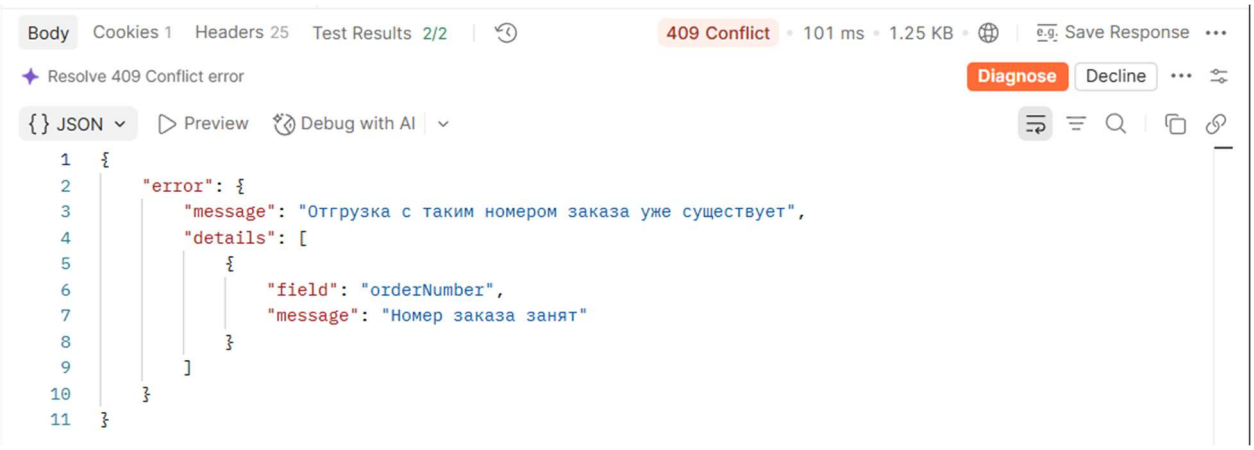


Рисунок 17 – Повторный номер заказа

Без токена отгрузки недоступны даже для чтения: код 401 (рисунок 18).

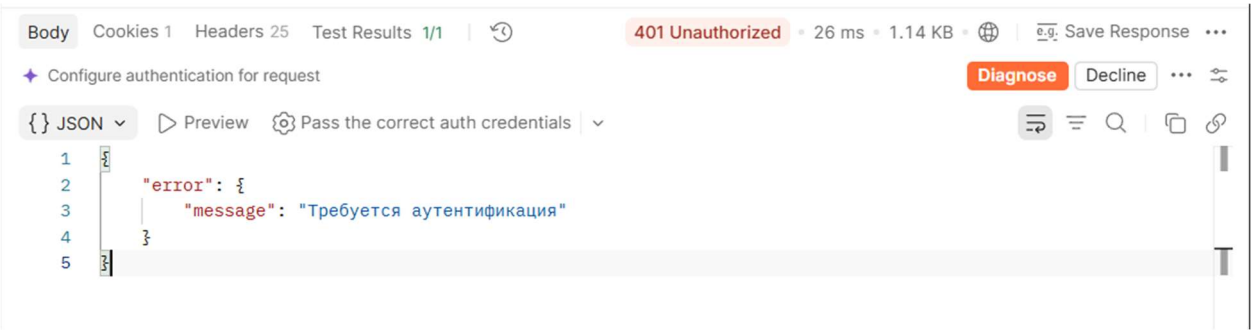


Рисунок 18 – Доступ без токена

Результат прогона всей коллекции показан на рисунке 22: 45 запросов, 68 проверок, одна ошибка. Коллекцию можно запускать повторно без очистки базы: номера заказов и *email* получают уникальный суффикс.

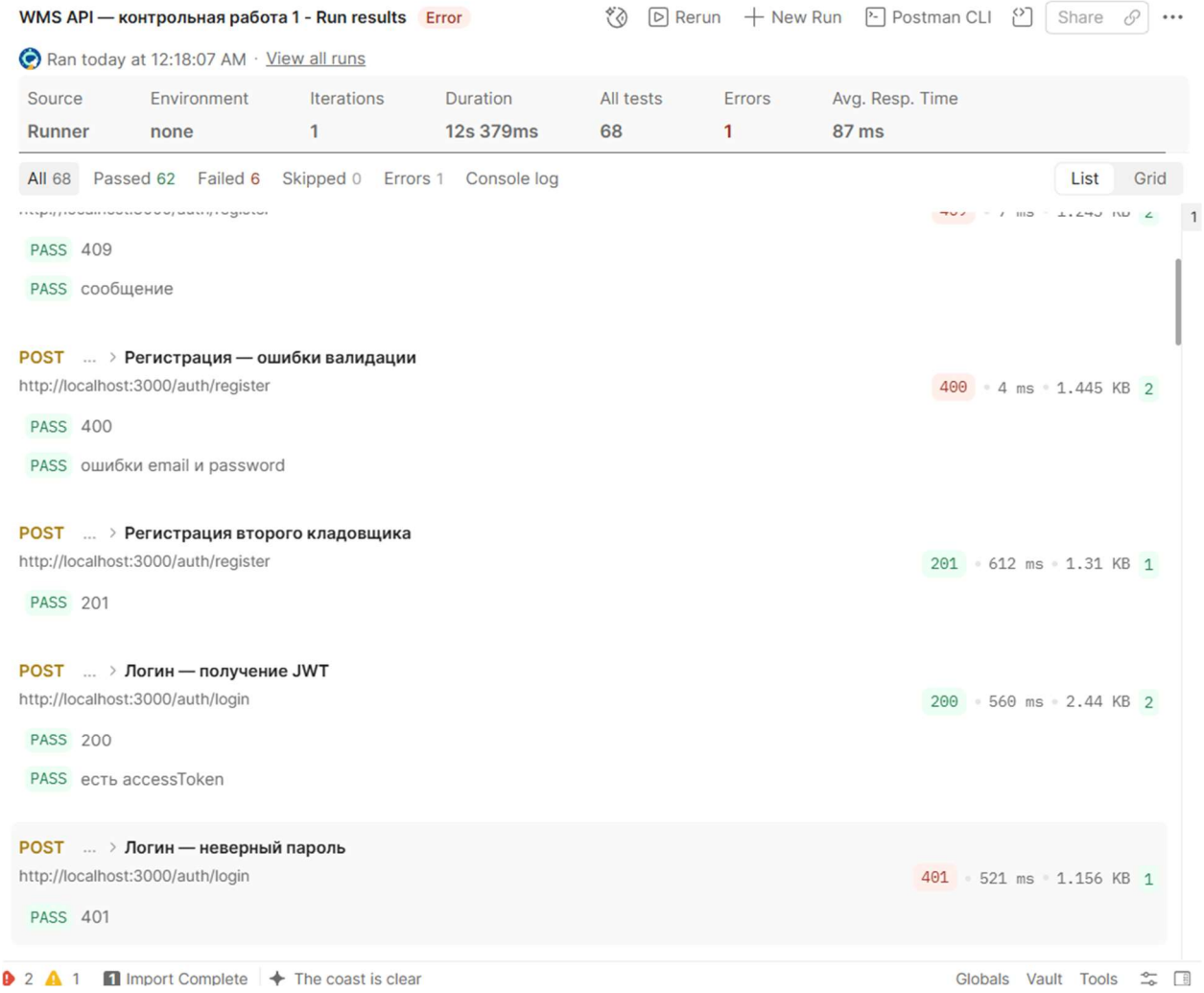


Рисунок 19 – Результат прогона коллекции

Кроме ручной проверки, написаны автотесты *Jest* и *Supertest* на отдельной базе *wms_test*: 36 тестов, из них 12 – на отгрузки: *CRUD*, права автора и администратора, доступ без токена, ошибки валидации, уникальность

номера заказа, переходы статусов, *QR*-код, фильтры, сортировку и статистику. Команда *npm test* проходит без ошибок.

7 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Какую сущность вы выбрали для реализации *CRUD* и как она связана с темой вашего курсового проекта?

Отгрузку (*Shipment*) – заказ покупателя, который склад собирает, упаковывает и отправляет. Тема курсового проекта – система управления складом и отгрузкой товаров с *QR*-кодами, поэтому отгрузка и её *QR*-код – центральная часть проекта: сервер выдаёт каждой отгрузке уникальный *QR*-код для этикетки, а терминал кладовщика находит отгрузку по отсканированному коду и меняет её статус.

2. Какие технологии вы использовали для разработки *API* и почему?

Node.js и *Express* 5 – простой и распространённый стек для *REST API*; *PostgreSQL* – реляционная база с транзакциями, уникальными индексами и внешними ключами; *Sequelize* – *ORM* с моделями, связями и параметризованными запросами; *jsonwebtoken* – *JWT*; *bcrypt* – хеширование паролей; *Joi* – декларативная валидация; *qrcode* – изображения *QR*-кодов; *helmet*, *express-rate-limit* и *cors* – защита; *Swagger UI* – документация; *Jest* и *Supertest* – автотесты; *Postman* – ручная проверка.

3. Как организована аутентификация? Опишите процесс регистрации и входа.

Регистрация *POST /auth/register*: *Joi* проверяет *email* и сложность пароля, сервер проверяет, что *email* свободен, хеширует пароль и создаёт пользователя с ролью *user*, ответ 201 без хеша.

Вход *POST /auth/login*: пользователь ищется по *email*, пароль сравнивается через *bcrypt.compare*. При успехе выдаётся *access*-токен *JWT* на 15 минут с *id* пользователя, ролью и *id* сессии, а *refresh*-токен ставится в *cookie HttpOnly*. Защищённые маршруты требуют заголовок *Authorization*: *Bearer <token>*, его проверяет *middleware authenticate*. После пяти неудачных попыток вход блокируется на 15 минут.

4. Как вы хешируете пароли? Почему это важно?

Пароль хешируется функцией *bcrypt.hash* с 12 раундами, в базе хранится только хеш в столбце *passwordHash*, по умолчанию он не выбирается из базы и не попадает в ответы. *bcrypt* добавляет случайную соль и работает медленно, поэтому при утечке базы пароли нельзя прочитать и трудно подобрать перебором, а одинаковые пароли дают разные хеши.

5. Какие роли пользователей вы реализовали и какие права доступа у каждой роли?

user (кладовщик) – просмотр отгрузок, создание, изменение своих, смена статуса, сканирование *QR*-кода, статистика; *moderator* (старший смены) – дополнительно изменение любых отгрузок и работа с ячейками; *admin* (администратор) – все права, включая удаление отгрузок, управление пользователями и ролями и журнал аудита. Без входа отгрузки недоступны. Роль при регистрации всегда *user*, изменить её может только администратор.

6. Как вы проверяете уникальность *email* при регистрации?

Валидатор приводит *email* к нижнему регистру и убирает пробелы. Затем сервис считает пользователей с таким *email* и при совпадении возвращает 409 «Пользователь с таким *email* уже существует». Дополнительно на столбце *users.email* есть уникальный индекс: если два запроса придут одновременно, второй отклонит база, и обработчик ошибок также вернёт 409. Так же проверяется уникальность номера заказа отгрузки.

7. Какие методы *HTTP* используются в вашем *API* и для каких операций?

GET – чтение списка, карточки, статистики, поиск по *QR*-коду и получение изображения *QR*-кода; *POST* – регистрация, вход и создание отгрузки; *PUT* – полная замена карточки отгрузки; *PATCH* – смена статуса; *DELETE* – удаление отгрузки администратором.

8. Как вы обрабатываете ошибки валидации? Приведите пример ответа.

Middleware validate проверяет данные схемой *Joi*, собирает все ошибки и передаёт обработчику ошибку с кодом 400. Ответ: {"*error*": {"*message*": "Ошибка валидации", "*details*": [{"*field*": "*plannedDate*", "*message*": "Плановая дата отгрузки не может быть в прошлом"}]}}. Занятые номер заказа или *email* и недопустимый переход статуса возвращают 409 в том же формате.

9. Как вы защитили *API* от *SQL*-инъекций (или *NoSQL*-инъекций)?

Все запросы выполняются через *Sequelize* с привязкой параметров, значения не склеиваются со строкой *SQL*. Параметры проходят *Joi*: *id* должен быть *UUID*, *QR*-код – соответствовать шаблону, даты – формату ГГГГ-ММ-ДД, поле сортировки выбирается только из белого списка, поэтому *sort=passwordHash* отклоняется кодом 400. Символы % и _ в поиске экранируются. Дополнительно строка запроса с *SQL*-шаблонами отклоняется до обработки.

10. Какие дополнительные меры безопасности вы применили (*helmet*, *rate limiting*, *CORS*)?

helmet – заголовки *Content-Security-Policy*, *X-Frame-Options*, *X-Content-Type-Options*; *express-rate-limit* – 100 запросов за 15 минут с *IP*, 20 на вход и регистрацию, 60 в минуту на пользователя; *CORS* по белому списку; лимит тела 100 КБ; приём только *JSON*; удаление *HTML* из строк; блокировка после пяти неудачных входов; короткий *access*-токен и *refresh*-токен в *cookie HttpOnly* с защитой от *CSRF*; журнал подозрительных действий и аудит удалений.

11. Как вы документировали *API*? Почему выбрали такой способ?

Swagger UI по описанию *OpenAPI* 3.0 на */api-docs* и файл *API.md*. *Swagger* позволяет посмотреть схемы и отправить запрос прямо из браузера, а описание в формате *OpenAPI* можно использовать для генерации клиента на фронтенде. *API.md* читается прямо на *GitHub* и содержит примеры запросов и ответов и таблицу переходов статусов.

12. Как вы тестировали *API*? Какие инструменты использовали?

Postman – коллекция из 45 запросов с проверками статусов и содержимого ответа, запускается через *Collection Runner* или *Newman* из командной строки. *Jest* и *Supertest* – 36 автотестов на отдельной базе: *CRUD*, права, валидация, статусы, *QR*-код, фильтры, статистика и меры защиты.

13. Какие статус-коды *HTTP* вы возвращаете при успешном создании, обновлении, удалении?

Создание – 201 *Created* с созданной записью; обновление *PUT* и смена статуса *PATCH* – 200 *OK* с обновлённой записью; удаление – 200 *OK* с сообщением. Ошибки: 400, 401, 403, 404, 409.

14. Как вы храните секретные данные (*JWT_SECRET*, пароль БД) в проекте?

В файле *.env*, который загружается пакетом *dotenv* и добавлен в *.gitignore*. В репозиторий попадает только *.env.example* с примерами. Секреты *JWT* генерируются случайно длиной 32 байта; сервер не запускается, если секрет короче 32 символов или секреты *access* и *refresh* совпадают. На хостинге переменные задаются в настройках сервиса.

15. Что такое *CORS*? Как вы его настроили для будущего фронтенда?

CORS – механизм браузера, который разрешает странице с одного источника обращаться к *API* на другом только при разрешающих заголовках сервера. В проекте разрешены источники из переменной *CORS_ORIGINS* (по умолчанию *http://localhost:5173* для фронтенда на *Vite*), методы *GET*, *POST*, *PUT*, *PATCH*, *DELETE*, заголовки *Content-Type*, *Authorization*, *X-CSRF-Token* и передача *cookie*. Запрос с другого источника получает 403.

16. Какие у вас планы по дальнейшему развитию этого *API* в курсовом проекте?

Добавить строки отгрузки со ссылками на товары и ячейки, резервирование и списание остатков при отправке, документы приёмки с *QR*-кодами грузовых мест, печать этикеток и журнал сканирований. В контрольной работе №3 к *API* будет подключён фронтенд: список отгрузок с фильтрами, статистика для начальника смены и экран сканирования *QR*-кода с камеры.

ВЫВОДЫ

Разработан *REST API* на *Node.js*, *Express* и *PostgreSQL* для основной сущности курсового проекта – отгрузки товаров со склада. Реализованы регистрация и вход с *JWT*, хеширование паролей *bcrypt*, роли *user*, *moderator* и *admin*, *CRUD* отгрузок с правилом «изменяет автор или администратор, удаляет только администратор», валидация *Joi* с проверкой уникальности номера заказа и *email* и проверкой дат.

Сверх обязательного *CRUD* реализованы статусы отгрузки с контролем переходов, выдача уникального *QR*-кода и его изображения для этикетки, поиск отгрузки по отсканированному коду, фильтрация, поиск, сортировка и статистика. Схема базы обновлена без потери данных, *API* описан в *Swagger* и *API.md*, секреты вынесены в *.env*. Работоспособность подтверждена коллекцией *Postman* из 45 запросов с 68 проверками и 36 автотестами *Jest*. Код размещён в ветке *kr21* и будет использован в следующих работах и в курсовом проекте. Код размещён в репозитории *GitHub*, ветка *kr21*: *https://github.com/mariaptk/wms-qr-system/tree/kr21*.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1 *Express – Node.js web application framework* [Электронный ресурс]. – Режим доступа: *https://expressjs.com*/. – Дата доступа: 30.10.2026.

2 *Sequelize v6 Documentation* [Электронный ресурс]. – Режим доступа: *https://sequelize.org/docs/v6*/. – Дата доступа: 30.10.2026.

3 *Jones*, *M*. *JSON Web Token* (*JWT*) : *RFC* 7519 [Электронный ресурс] / *M*. *Jones*, *J*. *Bradley*, *N*. *Sakimura*. – Режим доступа: *https://www.rfc-editor.org/rfc/rfc7519*. – Дата доступа: 30.10.2026.

4 *Joi API Reference* [Электронный ресурс]. – Режим доступа: *https://joi.dev/api*/. – Дата доступа: 30.10.2026.

5 *OpenAPI Specification v3.0.3* [Электронный ресурс]. – Режим доступа: *https://spec.openapis.org/oas/v3.0.3*. – Дата доступа: 30.10.2026.

6 *Postman Docs* [Электронный ресурс]. – Режим доступа: *https://learning.postman.com/docs*/. – Дата доступа: 30.10.2026.

7 *node-qrcode – QR code generator* [Электронный ресурс]. – Режим доступа: *https://github.com/soldair/node-qrcode*. – Дата доступа: 30.10.2026.

8 СТП 01-2024. Дипломные проекты (работы). Общие требования. – Минск : БГУИР, 2024.

ПРИЛОЖЕНИЕ А
(обязательное)
Листинги моделей, маршрутов, *middleware* и валидации

А.1 Модель пользователя (*src/models/user.js*)

```
const { DataTypes } = require('sequelize');
const { sequelize } = require('../config/database');

const User = sequelize.define('User', {
  id: {
    type: DataTypes.UUID,
    defaultValue: DataTypes.UUIDV4,
    primaryKey: true,
  },
  email: {
    type: DataTypes.STRING(255),
    allowNull: false,
    unique: true,
  },
  passwordHash: {
    type: DataTypes.STRING(255),
    allowNull: false,
  },
  role: {
    type: DataTypes.STRING(20),
    allowNull: false,
    defaultValue: 'user',
    validate: { isIn: [['user', 'moderator', 'admin']] },
  },
  name: {
    type: DataTypes.STRING(100),
    allowNull: true,
  },
  failedAttempts: {
    type: DataTypes.INTEGER,
    allowNull: false,
    defaultValue: 0,
  },
  lockUntil: {
    type: DataTypes.DATE,
    allowNull: true,
  },
}, {
  tableName: 'users',
  defaultScope: {
    attributes: { exclude: ['passwordHash'] },
  },
});

module.exports = User;
```

А.2 Модель отгрузки (*src/models/shipment.js*)

```
const { DataTypes } = require('sequelize');
const { sequelize } = require('../config/database');
const { SHIPMENT_STATUSES } = require('../constants/shipments');

const Shipment = sequelize.define('Shipment', {
  id: {
    type: DataTypes.UUID,
    defaultValue: DataTypes.UUIDV4,
    primaryKey: true,
  },
  orderNumber: {
    type: DataTypes.STRING(32),
    allowNull: false,
```

```
    unique: true,
  },
  qrCode: {
    type: DataTypes.STRING(32),
    allowNull: false,
    unique: true,
  },
  customerName: {
    type: DataTypes.STRING(200),
    allowNull: false,
  },
  deliveryAddress: {
    type: DataTypes.STRING(500),
    allowNull: false,
  },
  plannedDate: {
    type: DataTypes.DATEONLY,
    allowNull: false,
  },
  status: {
    type: DataTypes.STRING(16),
    allowNull: false,
    defaultValue: 'new',
    validate: { isIn: [SHIPMENT_STATUSES] },
  },
  packagesCount: {
    type: DataTypes.INTEGER,
    allowNull: false,
    defaultValue: 1,
    validate: { min: 1 },
  },
  totalWeightKg: {
    type: DataTypes.DECIMAL(10, 3),
    allowNull: true,
    // PostgreSQL возвращает DECIMAL строкой; клиенту отдаём число.
    get() {
      const value = this.getDataValue('totalWeightKg');
      return value === null || value === undefined ? value : Number(value);
    },
  },
  carrier: {
    type: DataTypes.STRING(100),
    allowNull: true,
  },
  comment: {
    type: DataTypes.TEXT,
    allowNull: true,
  },
  shippedAt: {
    type: DataTypes.DATE,
    allowNull: true,
  },
  createdBy: {
    type: DataTypes.UUID,
    allowNull: true,
  },
}, {
  tableName: 'shipments',
  indexes: [{ fields: ['status'] }, { fields: ['plannedDate'] }],
});

module.exports = Shipment;
```

A.3 Связи моделей (*src/models/index.js*)

```
const { sequelize } = require('../config/database');
const User = require('./user');
const Session = require('./session');
const Product = require('./product');
const StorageLocation = require('./storageLocation');
```

```
const AuditLog = require('./auditLog');
const Shipment = require('./shipment');


User.hasMany(Session, { foreignKey: 'userId', as: 'sessions', onDelete: 'CASCADE' });
Session.belongsTo(User, { foreignKey: 'userId', as: 'user' });


User.hasMany(Product, { foreignKey: 'createdBy', onDelete: 'SET NULL' });
Product.belongsTo(User, { foreignKey: 'createdBy', as: 'creator' });


User.hasMany(StorageLocation, { foreignKey: 'createdBy', onDelete: 'SET NULL' });
StorageLocation.belongsTo(User, { foreignKey: 'createdBy', as: 'creator' });


User.hasMany(Shipment, { foreignKey: 'createdBy', onDelete: 'SET NULL' });
Shipment.belongsTo(User, { foreignKey: 'createdBy', as: 'creator' });


User.hasMany(AuditLog, { foreignKey: 'userId', onDelete: 'SET NULL' });
AuditLog.belongsTo(User, { foreignKey: 'userId', as: 'actor' });


module.exports = {
  sequelize,
  User,
  Session,
  Product,
  StorageLocation,
  AuditLog,
  Shipment,
};
```

A.4 Статусы отгрузки и переходы (*src/constants/shipments.js*)

```
const SHIPMENT_STATUSES = Object.freeze(['new', 'picking', 'packed', 'shipped',
'delivered', 'cancelled']);


// Отгрузка движется только вперёд; отменить можно, пока груз не покинул склад.
const SHIPMENT_TRANSITIONS = Object.freeze({
  new: ['picking', 'cancelled'],
  picking: ['packed', 'cancelled'],
  packed: ['shipped', 'cancelled'],
  shipped: ['delivered'],
  delivered: [],
  cancelled: [],
});


// Пока груз на складе, карточку можно править; после отгрузки она становится
документом.
const SHIPMENT_OPEN_STATUSES = Object.freeze(['new', 'picking', 'packed']);


const SHIPMENT_SORT_FIELDS = Object.freeze([
  'createdAt',
  'plannedDate',
  'orderNumber',
  'customerName',
  'packagesCount',
  'totalWeightKg',
]);


module.exports = {
  SHIPMENT_STATUSES,
  SHIPMENT_TRANSITIONS,
  SHIPMENT_OPEN_STATUSES,
  SHIPMENT_SORT_FIELDS,
};
```

A.5 Маршруты аутентификации (*src/routes/auth.js*)

```
const express = require('express');
const { asyncHandler } = require('../utils/asyncHandler');
const { validate } = require('../middleware/validate');
const { authenticate } = require('../middleware/authenticate');
const { authLimiter, userLimiter } = require('../middleware/rateLimiters');
const { csrfGuard } = require('../middleware/security');
const { registerSchema, loginSchema } = require('../validators/auth');
const authController = require('../controllers/authController');


const router = express.Router();


router.post('/register', authLimiter, validate(registerSchema),
asyncHandler(authController.register));
router.post('/login', authLimiter, validate(loginSchema),
asyncHandler(authController.login));


router.post('/refresh', csrfGuard, asyncHandler(authController.refresh));
router.post('/logout', csrfGuard, asyncHandler(authController.logout));


router.get('/me', authenticate, userLimiter, asyncHandler(authController.me));
router.get('/sessions', authenticate, userLimiter,
asyncHandler(authController.sessions));
router.post('/logout-all', authenticate, userLimiter,
asyncHandler(authController.logoutAll));


module.exports = router;
```

A.6 Маршруты отгрузок (*src/routes/shipments.js*)

```
const express = require('express');
const { asyncHandler } = require('../utils/asyncHandler');
const { validate } = require('../middleware/validate');
const { authenticate } = require('../middleware/authenticate');
const { roleGuard } = require('../middleware/roleGuard');
const { userLimiter } = require('../middleware/rateLimiters');
const {
  idParams,
  qrParams,
  qrImageQuery,
  createShipmentSchema,
  replaceShipmentSchema,
  statusSchema,
  listShipmentsQuery,
} = require('../validators/shipment');
const shipmentController = require('../controllers/shipmentController');


const router = express.Router();


// В отгрузках имена и адреса получателей, поэтому без входа они не отдаются.
router.use(authenticate, userLimiter);


router.get('/', validate(listShipmentsQuery, 'query'),
asyncHandler(shipmentController.list));
// Объявлены раньше /:id, иначе «stats» и «scan» попадут в проверку UUID.
router.get('/stats', asyncHandler(shipmentController.stats));
router.get('/scan/:code', validate(qrParams, 'params'),
asyncHandler(shipmentController.scan));
router.get('/:id', validate(idParams, 'params'),
asyncHandler(shipmentController.getById));
router.get(
  '/:id/qr',
  validate(idParams, 'params'),
  validate(qrImageQuery, 'query'),
  asyncHandler(shipmentController.qrImage),
```

```
);

router.post(
  '/',
  roleGuard('user'),
  validate(createShipmentSchema),
  asyncHandler(shipmentController.create),
);
// Право автора проверяет контроллер: для этого нужна сама запись из базы.
router.put(
  '/:id',
  validate(idParams, 'params'),
  validate(replaceShipmentSchema),
  asyncHandler(shipmentController.replace),
);
router.patch(
  '/:id/status',
  validate(idParams, 'params'),
  validate(statusSchema),
  asyncHandler(shipmentController.changeStatus),
);
router.delete(
  '/:id',
  roleGuard('admin'),
  validate(idParams, 'params'),
  asyncHandler(shipmentController.remove),
);

module.exports = router;
```

А.7 *Middleware* аутентификации (*src/middleware/authenticate.js*)

```
const { User, Session } = require('../models');
const { AppError } = require('../utils/errors');
const { verifyAccess } = require('../utils/tokens');
const { logSecurity } = require('../utils/logger');

/**
 * Требует Authorization: Bearer <access JWT>.
 * Роль берётся из базы, а не из токена, чтобы смена роли действовала сразу.
 * Отозванная сессия гасит и ещё не истёкший access-токен.
 */
async function authenticate(req, _res, next) {
  const header = req.get('authorization') || '';
  const match = header.match(/^Bearer\s+(\S+)$/i);

  if (!match) {
    return next(new AppError(401, 'Требуется аутентификация'));
  }

  let payload;
  try {
    payload = verifyAccess(match[1]);
  } catch (err) {
    logSecurity('invalid_token', {
      ip: req.ip,
      path: req.originalUrl,
      requestId: req.id,
      reason: err.name,
    });
    return next(new AppError(401, 'Токен недействителен или истёк'));
  }

  const user = await User.findByPk(payload.sub);
  if (!user) {
    return next(new AppError(401, 'Требуется аутентификация'));
  }
```

```
  const session = await Session.findByPk(payload.sid);
  const sessionAlive = session
    && !session.revokedAt
    && session.expiresAt > new Date()
    && session.userId === user.id;

  if (!sessionAlive) {
    logSecurity('revoked_session', {
      ip: req.ip,
      path: req.originalUrl,
      requestId: req.id,
      userId: user.id,
    });
    return next(new AppError(401, 'Сессия недействительна'));
  }

  req.user = user;
  req.sessionId = session.id;
  return next();
}

module.exports = { authenticate };
```

A.8 Проверка роли (*src/middleware/roleGuard.js*)

```
const { ROLE_RANK } = require('../constants/roles');
const { AppError } = require('../utils/errors');
const { logSecurity } = require('../utils/logger');

/**
 * Пропускает пользователя, если его роль не ниже требуемой.
 * user < moderator < admin.
 * Отказ пишется в журнал подозрительных действий.
 */
function roleGuard(requiredRole) {
  if (!ROLE_RANK[requiredRole]) {
    throw new Error(`Неизвестная роль: ${requiredRole}`);
  }

  return (req, _res, next) => {
    if (!req.user) {
      return next(new AppError(401, 'Требуется аутентификация'));
    }

    if (ROLE_RANK[req.user.role] < ROLE_RANK[requiredRole]) {
      logSecurity('forbidden_role', {
        userId: req.user.id,
        role: req.user.role,
        requiredRole,
        method: req.method,
        path: req.originalUrl,
        ip: req.ip,
        requestId: req.id,
      });
      return next(new AppError(403, 'Недостаточно прав'));
    }

    return next();
  };
}

module.exports = { roleGuard };
```

A.9 *Middleware* валидации (*src/middleware/validate.js*)

```
const { AppError } = require('../utils/errors');

function formatJoi(error) {
  return error.details.map((detail) => ({
    field: detail.path.join('.') || detail.context?.key || 'value',
    message: detail.message,
  }));
}

/**
 * Проверяет тело, query или params схемой Joi.
 * stripUnknown отбрасывает поля, которых нет в схеме (защита от mass assignment).
 */
function validate(schema, source = 'body') {
  return (req, _res, next) => {
    const { error, value } = schema.validate(req[source], {
      abortEarly: false,
      stripUnknown: true,
      convert: true,
    });

    if (error) {
      return next(new AppError(400, 'Ошибка валидации', formatJoi(error)));
    }

    // В Express 5 req.query нельзя перезаписать: геттер каждый раз отдаёт исходную
строку.
    if (source === 'query') req.validQuery = value;
    else req[source] = value;
    return next();
  };
}

module.exports = { validate };
```

A.10 Схемы валидации регистрации и входа (*src/validators/auth.js*)

```
const Joi = require('joi');

const passwordRule = Joi.string()
  .min(8)
  .max(128)
  .pattern(/^(?=.*\d)(?=.*[^A-Za-z0-9]).+$/)
  .required()
  .messages({
    'string.min': 'Пароль должен содержать не менее 8 символов',
    'string.max': 'Пароль не должен быть длиннее 128 символов',
    'string.pattern.base': 'Пароль должен содержать цифру и спецсимвол',
    'any.required': 'Пароль обязателен',
    'string.empty': 'Пароль обязателен',
  });

const emailRule =
Joi.string().trim().lowercase().email().max(255).required().messages({
  'string.email': 'Некорректный email',
  'any.required': 'Email обязателен',
  'string.empty': 'Email обязателен',
});

const registerSchema = Joi.object({
  email: emailRule,
  password: passwordRule,
  name: Joi.string().trim().min(1).max(100).messages({
    'string.min': 'Имя не должно быть пустым',
```

```
    'string.max': 'Имя не должно быть длиннее 100 символов',
  }),
});

const loginSchema = Joi.object({
  email: emailRule,
  password: Joi.string().min(1).max(128).required().messages({
    'string.empty': 'Пароль обязателен',
    'any.required': 'Пароль обязателен',
  }),
});

module.exports = { registerSchema, loginSchema, passwordRule, emailRule };
```

A.11 Схемы валидации отгрузки (*src/validators/shipment.js*)

```
const Joi = require('joi');
const { idParams, paginationQuery } = require('./common');
const { SHIPMENT_STATUSES, SHIPMENT_SORT_FIELDS } = require('../constants/shipments');
const { today } = require('../utils/dates');

const DATE_PATTERN = /^\d{4}-\d{2}-\d{2}$/;

function isRealDate(value) {
  const date = new Date(`${value}T00:00:00Z`);
  return !Number.isNaN(date.getTime()) && date.toISOString().slice(0, 10) === value;
}

const dateOnly = Joi.string().pattern(DATE_PATTERN).custom((value, helpers) => (
  isRealDate(value) ? value : helpers.error('date.invalid')
)).messages({
  'string.pattern.base': 'Дата в формате ГГГГ-ММ-ДД',
  'date.invalid': 'Такой даты не существует',
});

const plannedDate = dateOnly.custom((value, helpers) => (
  value < today() ? helpers.error('date.past') : value
)).messages({
  'date.past': 'Плановая дата отгрузки не может быть в прошлом',
  'any.required': 'Плановая дата обязательна',
});

const orderNumber = Joi.string().trim().uppercase().pattern(/^[A-Z0-9][A-Z0-9-
]{2,31}$/).messages({
  'string.pattern.base': 'Номер заказа: 3–32 символа, латиница, цифры и дефис',
  'string.empty': 'Номер заказа обязателен',
  'any.required': 'Номер заказа обязателен',
});

const customerName = Joi.string().trim().min(2).max(200).messages({
  'string.empty': 'Получатель обязателен',
  'string.min': 'Получатель не короче 2 символов',
  'string.max': 'Получатель не длиннее 200 символов',
  'any.required': 'Получатель обязателен',
});

const deliveryAddress = Joi.string().trim().min(5).max(500).messages({
  'string.empty': 'Адрес доставки обязателен',
  'string.min': 'Адрес доставки не короче 5 символов',
  'string.max': 'Адрес доставки не длиннее 500 символов',
  'any.required': 'Адрес доставки обязателен',
});

const packagesCount = Joi.number().integer().min(1).max(1000).messages({
  'number.base': 'Количество мест должно быть числом',
```

```
  'number.integer': 'Количество мест – целое число',
  'number.min': 'Количество мест не меньше 1',
  'number.max': 'Количество мест не больше 1000',
});

const totalWeightKg =
Joi.number().min(0).max(50000).precision(3).allow(null).messages({
  'number.base': 'Вес должен быть числом',
  'number.min': 'Вес не может быть отрицательным',
  'number.max': 'Вес не больше 50 000 кг',
});

const carrier = Joi.string().trim().max(100).allow(null, '').messages({
  'string.max': 'Перевозчик не длиннее 100 символов',
});

const comment = Joi.string().trim().max(2000).allow(null, '').messages({
  'string.max': 'Комментарий не длиннее 2000 символов',
});

// status и qrCode с клиента не принимаются: статус меняется отдельным маршрутом, код
выдаёт сервер.
const createShipmentSchema = Joi.object({
  orderNumber: orderNumber.required(),
  customerName: customerName.required(),
  deliveryAddress: deliveryAddress.required(),
  plannedDate: plannedDate.required(),
  packagesCount: packagesCount.default(1),
  totalWeightKg,
  carrier,
  comment,
});

// PUT заменяет карточку целиком: обязательные поля те же, что при создании.
const replaceShipmentSchema = createShipmentSchema;

const statusSchema = Joi.object({
  status: Joi.string().valid(...SHIPMENT_STATUSES).required().messages({
    'any.only': `Статус: ${SHIPMENT_STATUSES.join(', ')}`,
    'any.required': 'Статус обязателен',
  }),
});

const qrParams = Joi.object({
  code: Joi.string().uppercase().pattern(/^SHP-[0-9A-F]{16}$/).required().messages({
    'string.pattern.base': 'QR-код отгрузки имеет вид SHP- и 16 шестнадцатеричных
символов',
  }),
});

const qrImageQuery = Joi.object({
  format: Joi.string().valid('png', 'svg').default('png').messages({
    'any.only': 'Формат: png или svg',
  }),
});

const listShipmentsQuery = Joi.object({
  ...paginationQuery,
  search: Joi.string().trim().max(100).allow(''),
  status: Joi.string().valid(...SHIPMENT_STATUSES).messages({
    'any.only': `Статус: ${SHIPMENT_STATUSES.join(', ')}`,
  }),
  plannedFrom: dateOnly,
  plannedTo: dateOnly.when('plannedFrom', {
    is: Joi.exist(),
    then: Joi.custom((value, helpers) => (
```

```
      value < helpers.state.ancestors[0].plannedFrom ? helpers.error('date.range') :
value
    )),
  }).messages({ 'date.range': 'plannedTo не может быть раньше plannedFrom' }),
  sort: Joi.string().valid(...SHIPMENT_SORT_FIELDS).default('plannedDate').messages({
    'any.only': `Сортировка: ${SHIPMENT_SORT_FIELDS.join(', ')}`,
  }),
  order: Joi.string().lowercase().valid('asc', 'desc').default('asc').messages({
    'any.only': 'Порядок: asc или desc',
  }),
});

module.exports = {
  idParams,
  qrParams,
  qrImageQuery,
  createShipmentSchema,
  replaceShipmentSchema,
  statusSchema,
  listShipmentsQuery,
};
```

A.12 Регистрация и вход, фрагмент (*src/services/authService.js*)

```
async function register({ email, password, name }) {
  // email уже приведён к нижнему регистру валидатором; уникальный индекс в базе
страхует от гонки.
  if (await User.unscoped().count({ where: { email } })) {
    throw new AppError(409, 'Пользователь с таким email уже существует', [
      { field: 'email', message: 'Email занят' },
    ]);
  }

  const passwordHash = await bcrypt.hash(password, config.bcryptRounds);
  const user = await User.create({
    email,
    passwordHash,
    name: name || null,
    role: 'user',
  });
  return user;
}

async function login({ email, password, ip, userAgent }) {
  const user = await User.unscoped().findOne({ where: { email } });

  if (user?.lockUntil && user.lockUntil > new Date()) {
    logSecurity('login_locked', { email, ip, userId: user.id });
    throw new AppError(423, LOCK_MESSAGE, { lockedUntil: user.lockUntil.toISOString()
});
  }

  const hash = user?.passwordHash || await warmDummyHash();
  const passwordOk = await bcrypt.compare(password, hash);

  if (!user || !passwordOk) {
    if (user) {
      user.failedAttempts += 1;
      if (user.failedAttempts >= config.lock.maxAttempts) {
        user.lockUntil = new Date(Date.now() + config.lock.lockMs);
        user.failedAttempts = 0;
        await user.save();
        logSecurity('login_locked', { email, ip, userId: user.id, reason:
'max_attempts' });
        throw new AppError(423, LOCK_MESSAGE, { lockedUntil:
user.lockUntil.toISOString() });
      }
```

```
      await user.save();
      logSecurity('login_failed', { email, ip, userId: user.id, failedAttempts:
user.failedAttempts });
    } else {
      logSecurity('login_failed', { email, ip, reason: 'unknown_user' });
    }
    throw new AppError(401, 'Неверный email или пароль');
  }

  user.failedAttempts = 0;
  user.lockUntil = null;
  await user.save();

  await enforceSessionLimit(user.id, ip);

  const session = await Session.create({
    userId: user.id,
    tokenHash: crypto.randomBytes(32).toString('hex'),
    expiresAt: new Date(Date.now() + config.jwt.refreshTtlMs),
    ip,
    userAgent: userAgent?.slice(0, 512) || null,
  });

  const refreshToken = signRefresh(user, session.id);
  session.tokenHash = hashToken(refreshToken);
  await session.save();

  return {
    user,
    accessToken: signAccess(user, session.id),
    refreshToken,
    csrfToken: newCsrfToken(),
    expiresIn: accessTtlSeconds(),
  };
}
```

A.13 Контроллер отгрузок (*src/controllers/shipmentController.js*)

```
const crypto = require('crypto');
const QRCode = require('qrcode');
const { Op, fn, col } = require('sequelize');
const { Shipment } = require('../models');
const { AppError } = require('../utils/errors');
const { logSecurity } = require('../utils/logger');
const { ROLE_RANK } = require('../constants/roles');
const {
  SHIPMENT_STATUSES,
  SHIPMENT_TRANSITIONS,
  SHIPMENT_OPEN_STATUSES,
} = require('../constants/shipments');
const { today } = require('../utils/dates');
const { writeAudit } = require('../services/auditService');

const EDITABLE_FIELDS = [
  'orderNumber',
  'customerName',
  'deliveryAddress',
  'plannedDate',
  'packagesCount',
  'totalWeightKg',
  'carrier',
  'comment',
];

function escapeLike(value) {
  return value.replace(/[\\%_]/g, '\\$&');
}
```

```
function newQrCode() {
  return `SHP-${crypto.randomBytes(8).toString('hex').toUpperCase()}`;
}


async function findShipment(id) {
  const shipment = await Shipment.findByPk(id);
  if (!shipment) throw new AppError(404, 'Отгрузка не найдена');
  return shipment;
}


async function assertOrderNumberFree(orderNumber, exceptId) {
  const where = { orderNumber };
  if (exceptId) where.id = { [Op.ne]: exceptId };
  if (await Shipment.count({ where })) {
    throw new AppError(409, 'Отгрузка с таким номером заказа уже существует', [
      { field: 'orderNumber', message: 'Номер заказа занят' },
    ]);
  }
}


/**
 * Менять карточку может её автор; старший смены и администратор — любую.
 */
function assertCanEdit(shipment, req) {
  const isAuthor = shipment.createdBy === req.user.id;
  if (isAuthor || ROLE_RANK[req.user.role] >= ROLE_RANK.moderator) return;


  logSecurity('forbidden_resource', {
    userId: req.user.id,
    role: req.user.role,
    resource: 'shipment',
    resourceId: shipment.id,
    method: req.method,
    path: req.originalUrl,
    ip: req.ip,
    requestId: req.id,
  });
  throw new AppError(403, 'Изменять отгрузку может только её автор или
администратор');
}


async function list(req, res) {
  const {
    page, limit, search, status, plannedFrom, plannedTo, sort, order,
  } = req.validQuery;
  const where = {};


  if (search) {
    const pattern = `%${escapeLike(search)}%`;
    where[Op.or] = [
      { orderNumber: { [Op.iLike]: pattern } },
      { customerName: { [Op.iLike]: pattern } },
      { deliveryAddress: { [Op.iLike]: pattern } },
    ];
  }
  if (status) where.status = status;
  if (plannedFrom || plannedTo) {
    where.plannedDate = {};
    if (plannedFrom) where.plannedDate[Op.gte] = plannedFrom;
    if (plannedTo) where.plannedDate[Op.lte] = plannedTo;
  }


  // sort и order уже проверены по белому списку, поэтому их можно подставить в ORDER
BY.
  const result = await Shipment.findAndCountAll({
```

```
    where,
    limit,
    offset: (page - 1) * limit,
    order: [[sort, order.toUpperCase()], ['createdAt', 'ASC'], ['id', 'ASC']],
  });

  res.json({
    data: result.rows,
    meta: {
      page,
      limit,
      total: result.count,
      pages: Math.ceil(result.count / limit),
      sort,
      order,
    },
  });
}

async function stats(_req, res) {
  const rows = await Shipment.findAll({
    attributes: [
      'status',
      [fn('COUNT', col('id')), 'count'],
      [fn('SUM', col('packagesCount')), 'packages'],
      [fn('SUM', col('totalWeightKg')), 'weightKg'],
    ],
    group: ['status'],
    raw: true,
  });

  const byStatus = Object.fromEntries(SHIPMENT_STATUSES.map((status) => [status, 0]));
  let total = 0;
  let packages = 0;
  let weightKg = 0;
  for (const row of rows) {
    const count = Number(row.count);
    byStatus[row.status] = count;
    total += count;
    if (row.status !== 'cancelled') {
      packages += Number(row.packages || 0);
      weightKg += Number(row.weightKg || 0);
    }
  }

  const date = today();
  const open = { [Op.in]: SHIPMENT_OPEN_STATUSES };
  const [overdue, plannedToday] = await Promise.all([
    Shipment.count({ where: { status: open, plannedDate: { [Op.lt]: date } } }),
    Shipment.count({ where: { status: open, plannedDate: date } }),
  ]);

  res.json({
    data: {
      total,
      byStatus,
      plannedToday,
      overdue,
      packages,
      weightKg: Math.round(weightKg * 1000) / 1000,
    },
  });
}

async function getById(req, res) {
  res.json({ data: await findShipment(req.params.id) });
}
```

```
async function scan(req, res) {
  const shipment = await Shipment.findOne({ where: { qrCode: req.params.code } });
  if (!shipment) throw new AppError(404, 'Отгрузка с таким QR-кодом не найдена');
  res.json({ data: shipment });
}


async function qrImage(req, res) {
  const shipment = await findShipment(req.params.id);
  const options = { errorCorrectionLevel: 'M', margin: 2, width: 300 };


  if (req.validQuery.format === 'svg') {
    res.type('image/svg+xml').send(await QRCode.toString(shipment.qrCode, {
...options, type: 'svg' }));
    return;
  }
  res.type('image/png').send(await QRCode.toBuffer(shipment.qrCode, options));
}


async function create(req, res) {
  await assertOrderNumberFree(req.body.orderNumber);
  const shipment = await Shipment.create({
    ...req.body,
    status: 'new',
    qrCode: newQrCode(),
    createdBy: req.user.id,
  });
  res.status(201).json({ data: shipment });
}


async function replace(req, res) {
  const shipment = await findShipment(req.params.id);
  assertCanEdit(shipment, req);
  if (!SHIPMENT_OPEN_STATUSES.includes(shipment.status)) {
    throw new AppError(409, `Отгрузку в статусе ${shipment.status} изменить нельзя`);
  }
  await assertOrderNumberFree(req.body.orderNumber, shipment.id);


  for (const field of EDITABLE_FIELDS) {
    shipment[field] = req.body[field] ?? null;
  }
  await shipment.save();
  res.json({ data: shipment });
}


async function changeStatus(req, res) {
  const shipment = await findShipment(req.params.id);
  const from = shipment.status;
  const to = req.body.status;
  const allowed = SHIPMENT_TRANSITIONS[from];


  if (!allowed.includes(to)) {
    throw new AppError(409, `Нельзя перевести отгрузку из статуса ${from} в ${to}`, {
from, to, allowed });
  }


  shipment.status = to;
  if (to === 'shipped') shipment.shippedAt = new Date();
  await shipment.save();
  res.json({ data: shipment });
}


async function remove(req, res) {
  const shipment = await findShipment(req.params.id);
  await shipment.destroy();
```

```
  await writeAudit({
    userId: req.user.id,
    action: 'shipment.deleted',
    entity: 'shipment',
    entityId: shipment.id,
    details: { orderNumber: shipment.orderNumber, status: shipment.status },
    req,
  });
  res.json({ data: { message: 'Отгрузка удалена' } });
}

module.exports = {
  list,
  stats,
  getById,
  scan,
  qrImage,
  create,
  replace,
  changeStatus,
  remove,
};
```